

报名序号:003158

论文题目：光伏电站发电功率日前预测问题

光伏电站发电功率日前预测问题

摘要

面对光伏电站大规模并网背景下的功率波动性与调度不确定性问题,本文基于历史功率数据与数值天气预报信息,构建了一套多层次、组合型建模框架,聚焦日前(24-48小时)光伏功率预测任务,在保证物理解释力的前提下,系统提升了预测精度与模型鲁棒性,全文依次针对光伏功率的周期性、扰动性、多源信息融合及气象数据空间降尺度展开建模与验证,旨在为新能源并网提供更高质量的功率预测支持。

针对问题一,本文基于太阳辐射物理机制,结合电站地理参数,构建了理论发电功率模型,并与实测功率对比分析,揭示出光伏发电中显著的季节性与日内周期性变化特征,量化了高温、云层遮挡等典型气象因素对发电输出的扰动影响。

针对问题二,利用历史发电功率序列,构建多种深度学习预测模型(PureLSTM、FusionModel、BiFusionModel),通过滑动窗口与特征分解方法提取序列特征,预测未来一日15分钟粒度的功率曲线。结果表明,FusionModel在不引入气象信息的情况下已具备较强的时序拟合与泛化能力。

针对问题三,在原模型基础上引入NWP气象预报数据,构建多源融合模型,并对比不同天气场景(晴天、多云、阴雨)下的预测性能差异。实验结果表明,FusionModel在所有场景下均优于单一时序模型,尤其在光照稳定、高温的夏季天气中表现最为优越,其RMSE较问题二显著下降约21.9%。

针对问题四,进一步研究NWP空间分辨率对预测精度的影响,引入实测LMD气象数据,并通过混合输入方式(NWP+LMD)进行模型训练。结果显示,使用高分辨率LMD数据可将预测RMSE降低至0.6576,显著优于仅使用NWP数据(RMSE=0.8359),证明降尺度处理对提升局地天气感知能力具有显著作用。

通过上述系统建模与对比分析,本文最终构建了一套具备强泛化能力与高预测精度的光伏电站功率预测体系,并提出了基于场景识别的模型选择策略,为提升新能源电力系统的调度稳定性与资源利用效率提供了技术支持。

关键词: 光伏功率预测; 深度学习; 多源融合; 空间降尺度; 天气场景划分

1. 问题背景与重述

1.1. 问题背景

光伏发电是一种将太阳能直接转化为电能的清洁能源方式，具有绿色环保、可持续等优点。随着光伏电站的大规模接入电网，其发电输出的不稳定性也逐渐成为电网调度的难题。由于光照受季节、天气影响较大，光伏功率存在明显的波动性和不确定性。为了保障电网运行的平稳性，调度中心需要提前预测未来 24 到 48 小时的光伏发电功率，这被称为“日前预测”。然而，传统预测方法难以充分应对复杂的气象变化，因此如何提高预测精度成为一个关键技术问题。

本题需结合光伏电站的历史发电数据和数值天气预报（NWP）信息，研究建模方法以提升预测准确性，并探讨天气、地理、季节等因素对模型表现的影响，为新能源消纳与电网调度提供数据支撑与方法参考。

1.2. 问题重述

光伏发电功率受气象条件影响显著，输出具有强波动性与不确定性。为实现高比例新能源并网并保障电网稳定，需提升日前预测（24-48 小时）精度。本文需结合历史功率数据与数值天气预报（NWP），解决以下问题：

1.2.1. 问题一：光伏发电的特性分析

基于地理位置参数（经纬度、海拔等）构建理论发电功率模型，与实际功率对比，分析季节尺度（长周期）与日内尺度（短周期）的变化规律，量化阴天、降雨等气象扰动对功率波动的影响。

1.2.2. 问题二：基于历史功率的预测模型构建

仅利用历史功率数据，建立未来 7 天、15 分钟粒度的日前预测模型，使用第 2、5、8、11 月最后一周作为测试集评估误差，重点优化白昼时段预测精度。

1.2.3. 问题三：融合 NWP 信息的预测模型构建与分析

在问题二基础上引入 NWP 气象数据（辐射、温度等），构建多源融合模型，对比其在天气、季节差异下的性能提升效果，划分适用 NWP 的典型场景（如晴天、多云）并进行验证。

1.2.4. 问题四：探讨 NWP 空间降尺度技术的作用

针对传统 NWP 数据空间分辨率不足的问题，通过机器学习或统计建模进行降尺度处理，验证其对局部气象表征能力及预测精度的提升作用，明确其适用条件与优势。

综上所述，本题要求我们系统分析光伏发电的特性，构建多种预测模型，并引入气象数据与数据处理技术，不断提升预测精度。最终目标是为光伏电站的运行调度提供科学、可靠的功率预测方案。

2. 问题分析

2.1. 问题一：光伏发电特性分析

光伏发电的核心驱动力是太阳辐射，受天文位置和气象条件双重影响。本问题要求根据电站的地理位置和太阳高度角等因素，构建理论可发功率模型，并与实际观测功率数据进行对比分析。重点是揭示发电过程中：

1. 由季节性太阳辐照变化引起的长周期变化；
2. 由日照间歇、云层遮挡等造成的短周期波动；
3. 识别实际功率与理论可发功率之间不同的偏差，这些偏差能反映系统效率损耗或气象扰动，分析这些偏差对建模有重要参考价值。

2.2. 问题二：基于历史功率的预测模型构建

在问题 1 的基础上，本问题需要构建依赖历史功率序列的光伏功率日前预测模型，在不考虑未来气象条件的情况下，充分挖掘时间序列本身的内在规律。需考虑时间序列中的周期性、趋势性及随机性特征，采用合适的统计模型或机器学习方法对未来短期（24~48 小时）发电功率进行预测。通过与真实观测值的对比，评估模型的预测性能与泛化能力，同时也将问题二构建的模型作为后续复杂模型的参照基础。

2.3. 问题三：融合 NWP 信息的预测模型构建

相较于问题 2，本问题引入了数值天气预报（NWP）数据，需要构建融合气象信息的多源预测模型，以增强模型对未来天气变化的响应能力。重点在于分析气温、辐射、云量等关键气象因子与发电功率之间的关系，并将其有效嵌入预测框架中。通过与问题二构建的模型进行对比分析，验证 NWP 信息的引入是否在整体或特定场景下显著提高了预测精度，同时需要考虑不同天气/季节场景下模型的预测表现差异，并对其适用性进行场景划分与归纳总结，并据此提出不同场景下的模型选用策略。

2.4. 问题四：探讨 NWP 空间降尺度对预测精度的影响

本问题进一步探讨传统 NWP 数据空间分辨率与光伏电站空间尺度不匹配所带来的预测偏差。考虑到 MW 级光伏电站的地理覆盖面积较小，而 NWP 数据通常为千米级网格，空间分辨率一般在 $0.25^{\circ} \times 0.25^{\circ}$ 或更粗粒度，可能不足以准确描述一个光伏电站所处的小气候特征。为提升预测精度，本问题需要我们引入空间降尺度方法，获取更细尺度的气象输入。最终，通过与问题 3 预测结果的对比，检验降尺度方法在精度提升方面的有效性，并分析其适用条件与影响机制，评估其在光伏功率预测中的实用潜力。

3. 模型假设

- 3.1. 假设 1：光伏电板的倾斜角度与发电站的纬度相同。
- 3.2. 假设 2：若发电功率大于 0.05W 则为白天。
- 3.3. 假设 3：数据集的观测周期内光伏系统组件未发生硬件变更或运行参数调整。

- 3.4. 假设 4：忽略组件老化及灰尘遮挡等长期退化因素。
- 3.5. 假设 5：站点位于 NWP 数据采集区域的几何中心，并且其数据覆盖范围呈圆形。

4. 符号定义和说明

表 1 符号定义和说明	
符号	符号说明
P	功率（W）
$GHI(t)$	时刻 t 的水平面总辐照度（W/m ² ）
$DNI(t)$	时刻 t 的法向直射辐照度（W/m ² ）
$DHI(t)$	时刻 t 的水平面散射辐照度（W/m ² ）
$P(t)$	时刻 t 的大气压强（hPa）
$T(t)$	修正因子
$U_w(t)$	时刻 t 的可降水量（cm）

若后文出现表中未有符号，会在相应位置详细说明

5. 数据说明及处理

5.1. 数据来源说明

针对问题一，采用湖北麻城 110MW 光伏电站两年期数据（15 分钟采样），该站地处中部丘陵多气候区，数据涵盖完整年度周期性变化，适于分析长时序规律与气象扰动影响；问题二至四选用河北某 6600KW 电站一年期数据（东经 117.45722°、北纬 38.18306°），该站位于中纬度太阳能富集区，数据集包含 NWP 气象预报与 LMD 实测数据，时间分辨率 15 分钟。太阳能电站的传感器架构和数据收集过程如下图所示：

太阳能电站的传感器架构和数据收集过程如下图所示：

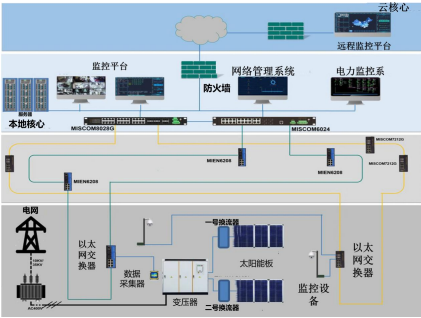


图 1 传感器架构和数据收集过程图

5.2. 数据基本信息展示

表 2 数据基本信息展示表

数据规格	参数值
数据来源（数据集链接）	https://www.scidb.cn/en/detail?dataSetId=f8f3d7af144f441795c5781497e56b62
光伏电站装机容量	20000MW
发电功率和 NWP 数据时间分辨率	采样点/15min
发电功率和 NWP 数据起始-截至时间（一年）	2018.06.30-2019.06.30
NWP 属性	全球水平辐射、直射辐射、气温、湿度、风速、风向、气压

本研究采用的数据集关键特征如表所示（部分示例）。鉴于数据维度高、特征变量复杂，为提升可读性与分析效率，将原始数据矩阵进行行列转置，即将样本记录作为行索引，特征属性作为列标题。

表 3 部分数据集展示表

date_time	2018/6/3016:00	2018/6/3016:15	2018/6/3016:30
nwp_globalirrad	0	0	0
nwp_directirrad	0	0	0
nwp_temperature	26.47	26.24	26.03
nwp_humidity	54.44	55.63	56.73
nwp_windspeed	3.66	3.53	3.38
nwp_winddirection	127.96	128.51	128.77
nwp_pressure	1004.46	1004.45	1004.52
lmd_totalirrad	0	0	0
lmd_diffuseirrad	0	0	0
lmd_temperature	24.700001	24.5	24.700001
lmd_pressure	1007.2000119	1007.2000119	1006.099976
lmd_winddirection	161.399994	147.300003	144
lmd_windspeed	0.9	1.3	0.7
power	0	0	0

5.3. 数据预处理

5.3.1. 数据归一化

由于模型输入特征来源多样，变量单位及数值范围存在显著差异，若不统一处理将影响模型的梯度传播与收敛速度。为提升模型训练稳定性，本文对所有连续型输入变量进行了 min-max 归一化处理，将各变量线性映射至[0,1]区间。归一化公式如下

$$x' = \frac{x - x_{\min}}{x_{\max} - x_{\min}}$$

其中 x 为原始数值， $x_{\min}$ 、 $x_{\max}$ 分别为训练集中该变量的最小值与最大值。此外，在模型输出阶段，为便于误差评估，模型预测结果将经反归一化还原为实际功率单位（MW）。

5.4. 数据特征统计与相关性分析

5.4.1. 特征变量统计描述

这一部分我们依次从年尺度趋势、月内辐照变化和月均功率变化展开，明确 GHI 作为主要影响因子的主导作用；随后从日内平均行为与高频热力结构进一步刻画发电的周期性变化特征；最终结合效率和温度的关系揭示非线性物理限制，为后续特征工程与建模策略提供理论支持。

1. 时间尺度趋势分析

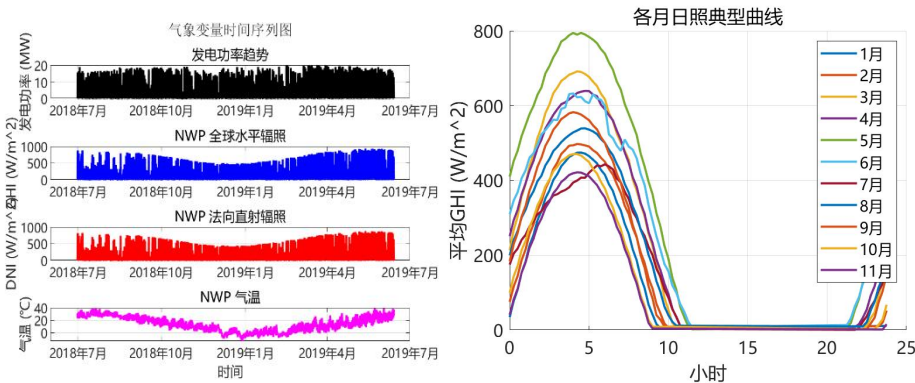


图 2 气象变量时间序列图

图 2 各月典型日照模式

光伏电站的发电功率与辐照、气温等变量在年、月尺度上呈现显著的周期性变化特征。GHI 与 DNI 的年度序列显示，夏季辐照强度达到峰值，冬季显著下降，与发电功率的季节性波动高度一致。进一步分析各月典型日照模式，夏季日照时长显著延长，且正午辐照强度可达 $1000 W/m^2$ 以上，而冬季日照时长缩短至 8 - 9 小时，辐照强度波动范围收窄。

2. 短时周期性分析

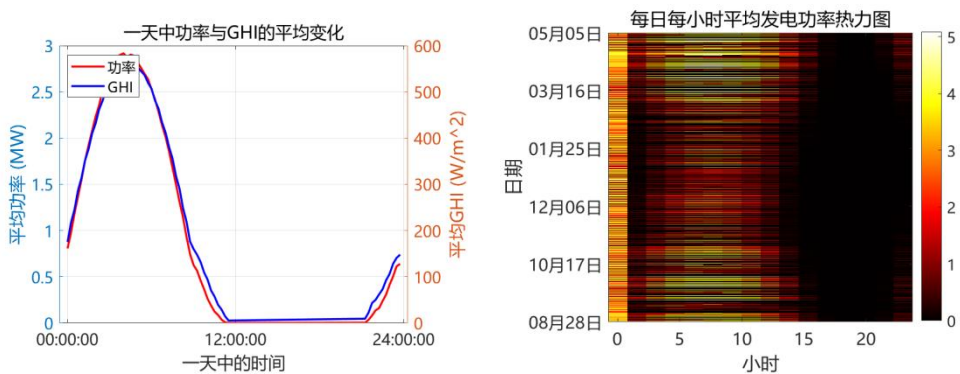


图 4 日内发电功率与辐照强度

图 5 逐小时功率分布热力图

日内发电功率与辐照强度呈现典型的“钟形曲线”特征（图 3），两者同步性显著。早间 7:00 至 8:00 为功率快速爬升阶段，正午 12:00 - 14:00 达到峰值，随后逐步下降，至 18:00 后趋近于零。

逐小时功率分布热力图进一步显示，夏季（3 - 9 月）日间功率持续高位运行，且波动较小；冬季则因日照时间缩短，功率活跃期压缩至 9:00 - 16:00。值得注意的是，多云天气下功率曲线呈现高频波动导致功率在 15 分钟内骤降 30% - 50%，此类短时扰动需通过高频特征建模捕捉。

3.多变量协同效应分析

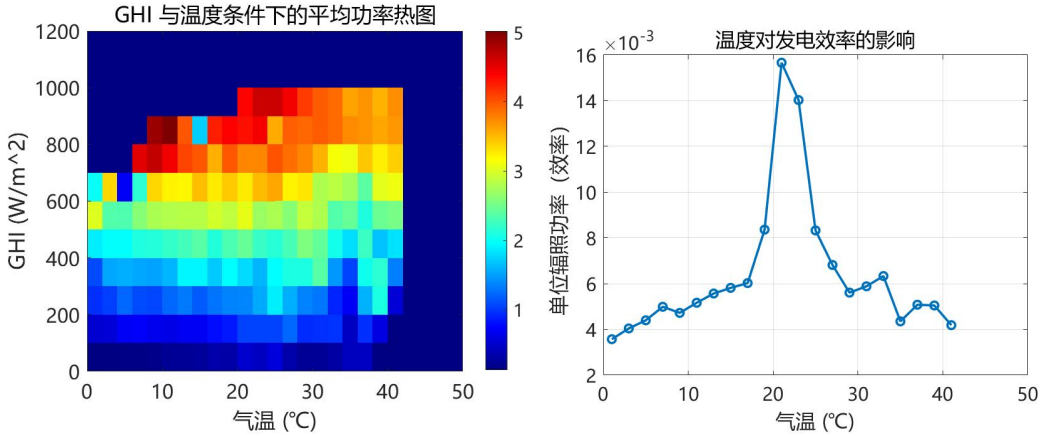


图 6 GHI 与温度条件下的平均功率热力图 图 7 温度对发电效率的影响

功率与 NWP、LMD 呈强正相关，而与湿度呈弱负相关。温度对功率的影响具有阈值效应：当气温低于 20°C 时，功率随温度升高线性增长；超过 30°C 后，因组件效率下降，功率增幅趋缓甚至下降。进一步分析不同 GHI 气温组合下的功率分布，发现高辐照与中低温条件下功率输出最大，而高温或低辐照场景功率显著受限。此外，风速、气压等变量与功率的线性相关性微弱，但其潜在的非线性作用需通过深度模型进一步挖掘。

5.4.2. 特征间相关性分析

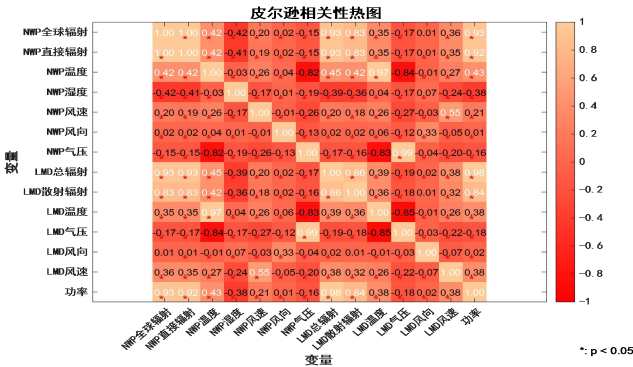


图 8 皮尔逊相关性热力图

为深入探究输入特征变量与光伏发电功率的关联性，本文通过皮尔逊相关系数检验及热力图分析揭示了变量间的线性相关规律。结果表明，辐照变量与功率呈现显著正相关，温度变量与功率呈中等正相关，表明温度可能通过影响组件效率或与辐照协同波动间接调节输出功率。相比之下，风速、风向及气压等变量与功率的线性相关性微弱。此外，NWP 与 LMD 数据在温度、辐射等关键参数上呈现高度一致性。

6. 模型建立与求解

6.1. 问题 1 模型的建立与求解

6.1.1. 建模思路

光伏电站的发电性能受太阳辐照度、气象条件及地理因素的综合影响。为有效评估系统在理想辐照条件下的最大发电能力,本文构建了一个**物理驱动的理论功率模型** $P_{\text{theo}}(t)$,用于表示在标准化假设条件下,光伏电站在时刻 t 应能达到的理论最大输出功率(单位: MW)。该模型不仅可用于评估实际运行效率、识别系统性能波动和气象敏感性,还可用于量化其长期(季节性)和短期(日内)功率变化特征,并深入解析偏差产生的气象机制。以下是关键分析步骤及思路:

理论功率模型的总体结构如公式所示:

$$P_{\text{theo}}(t) = \eta(t) \cdot G_{\text{eff}}^*(t) \cdot A$$

其中, $\eta(t)$ 为考虑温度影响后组件在 t 时刻的转换效率; $G_{\text{eff}}^*(t)$ 为考虑了入射角、漫反射、地面反射以及物种大气透射率修正后的有效辐照度; A 为光伏组件总面积,单位为 m^2 。该表达式体现了理论功率受组件效率、有效辐照强度及装置规模的共同影响。下文逐一对模型中的核心参数进行建模。

1. 光伏组件的总面积 A 的估计方法

根据光伏系统的设计原则,其额定装机容量 P_{nominal} 可由组件总面积 A 与单位面积最大输出功率 P_{unit} 的乘积给出: $P_{\text{nominal}} = A \cdot P_{\text{unit}}$,其中 $P_{\text{unit}} = \eta \cdot 1000\text{W}/\text{m}^2$ 。设参考效率 $\eta = 0.18$,则: $P_{\text{unit}} = 180\text{W}/\text{m}^2$,问题 1 所使用数据集系统设计容量为 110 MW,则所需光伏组件总面积为: $A = \frac{110 \times 10^6}{180} \approx 611111.11\text{m}^2$ 。

2. 光伏组件效率变化建模

组件工作效率受温度显著影响,尤其是在高辐照条件下组件表面温升明显。由于电池片温度难以直接获得,本文采用以下经验公式,结合气象数据对效率进行动态估计:

$$\eta(t) = \eta_{\text{ref}} \cdot [1 - \gamma \cdot (T_{\text{air}}(t) + \kappa \cdot GHI(t) - 25)]$$

其中:

- $\eta_{\text{ref}} = 0.18$: 标准测试条件下的参考转换效率;
- $\gamma = 0.0045$: 温度影响系数;
- $T_{\text{air}}(t)$: 时刻 t 的环境空气温度;
- $\kappa = 0.03$: 辐照度引起温升的经验系数;
- $GHI(t)$: 全水平面太阳总辐照度;
- 25 为 STC (标准测试条件) 下的基准温度 (单位: $^{\circ}\text{C}$)。

该公式综合考虑了空气温度与辐照引起的组件温升对效率的动态调制作用。

3. 有效辐照度建模

在光伏发电系统中,电池板的发电功率高度依赖其表面接收到的太阳辐射能量。然而,太阳辐射到达地表后会被大气散射、反射和吸收,其在组件表面的能量并非单一来源,而是由多种辐射分量叠加而成,因此本文建立以下的模型,将不同辐射分量相加,实际光伏组件接收到的辐照强度由直射、漫反射和地面反射三个部分组成。有效辐照度模型如下:

$$G_{\text{eff}}(t) = DNI(t) \cdot \cos(\theta_i(t)) + DHI(t) \cdot \frac{1 + \cos(\beta)}{2} + \rho_g \cdot GHI(t) \cdot \frac{1 - \cos(\beta)}{2}$$

其中：

- $DNI(t)$: 法向直射辐照度；
- $DHI(t)$: 水平面散射辐照度；
- $GHI(t)$: 总水平辐照度；
- $\theta_i(t)$: 太阳入射角；
- β : 组件倾角；
- ρ_g : 地表反射率（一般取 0.2 ~ 0.3）。

入射角计算公式为：

$$\cos(\theta_i(t)) = \cos(\theta_z(t))\cos(\beta) + \sin(\theta_z(t))\sin(\beta)\cos(\phi_s(t) - \phi_p)$$

其中 $\theta_z(t)$ 为太阳天顶角， $\phi_s(t)$ 为太阳方位角， ϕ_p 为组件方位角。

4. 大气透射率修正因子计算

在光伏理论功率建模中，为了更准确地评估光伏组件表面实际接收到的太阳辐射强度，需要考虑大气层对辐射的衰减效应。太阳辐射在穿过大气层的过程中，会受到分子散射、气溶胶散射、臭氧吸收、水汽吸收及混合气体吸收等多种机制的影响。因此，本文引入大气透射率修正因子对有效辐照度进行修正：

$$G_{\text{eff}}^*(t) = T_r(t) \cdot T_a(t) \cdot T_o(t) \cdot T_w(t) \cdot T_g(t) \cdot G_{\text{eff}}(t)$$

各透射率分项含义及表达如下： - 瑞利散射透射率 $T_r(t)$:

$$T_r(t) = \exp\left(-0.0903 \cdot \left(\frac{P(t)}{1013.25}\right)^{0.84} \cdot (1 + \cos(\theta_z(t)))^{-1.01}\right)$$

- 气溶胶透射率 $T_a(t)$:

$$T_a(t) = \exp\left(-\tau_a \cdot (0.6777 + 0.1464\tau_a - 0.00626\tau_a^2) \cdot m_a(t)\right)$$

- 臭氧吸收透射率 $T_o(t)$:

$$T_o(t) = 1 - \frac{0.011 \cdot U_o \cdot m_o(t)}{1 + 0.006 \cdot (U_o \cdot m_o(t))^{1.5}}$$

- 水汽吸收透射率 $T_w(t)$:

$$T_w(t) = 1 - 0.077 \cdot (U_w(t) \cdot m_w(t))^{0.3}$$

- 混合气体透射率 $T_g(t)$:

$$T_g(t) = \exp(-0.0117 \cdot m_g(t)^{0.3139})$$

综上所述，最终理论功率模型形式如下：

$$P_{\text{theo}}(t) = \eta_{\text{ref}} \cdot [1 - \gamma \cdot (T_{\text{air}}(t) + \kappa \cdot GHI(t) - 25)] \cdot T_r(t) \cdot T_a(t) \cdot T_o(t) \cdot T_w(t) \cdot T_g(t) \cdot \left[DNI(t) \cos(\theta_i(t)) + DHI(t) \cdot \frac{1 + \cos(\beta)}{2} + \rho_g \cdot GHI(t) \cdot \frac{1 - \cos(\beta)}{2} \right] \cdot A$$

该模型兼具物理真实性与可操作性，可为光伏系统的性能评估、偏差诊断及气象适应性分析提供理论依据。

6.1.2. 模型计算结果

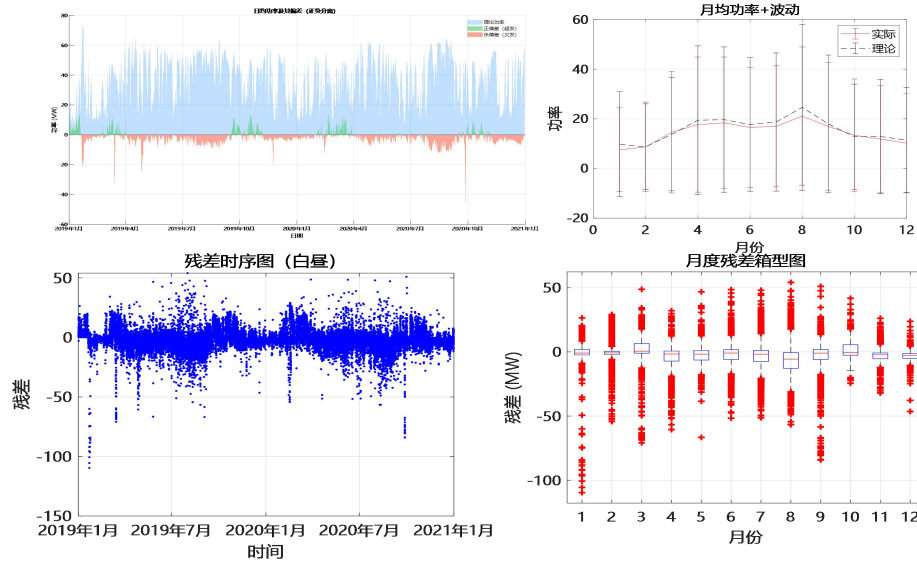


图 9 理论功率计算图

通过 MATLAB 实现光伏发电理论功率模型，计算得出理论可发功率模型评估其运行绩效，选取日均功率及其偏差、月均功率+波动、残差时序图（白昼）、月度残差箱线图。该典型的四类对比与偏差图示分别刻画了系统在长周期（季节）与短周期（日内）的响应特性及残差分布情况，有助于分别刻画了系统在长周期（季节）与短周期（日内）的响应特性及残差分布情况。

长周期季节性变化分析：

- 日均功率及其偏差展现出典型的季节性特征，夏季高、冬季低，符合太阳辐射角度年度周期变化；
- 实际功率长期偏低，尤其在春夏高辐照月份尤为明显，暗示系统在峰值运行能力受限，即实际工况中会收到除模型之外的设备老化突发事故等情况影响；
- 个别时段出现较大正偏差（绿色区域），可能源于模型在阴天误估辐照或逆变器电力滞后性未能准确表征。
- 月均功率+波动进一步对月均功率进行统计。理论与实际均呈现“单峰型”年周期变化，最大值出现在 7~8 月。然而实际功率误差的标准差远大于理论值，反映出实际发电受天气和设备运行状态的影响更加剧烈。这种强烈的波动性对于调度系统构成挑战。

短周期日内波动与偏差特征：

- 残差时序图(白昼)显示了白昼时段残差的时间序列分布，残差定义为“实际 - 理论”。可见大部分数据集中于负值区域，即大多数时间段内实际

功率低于理论预期。

- 月度残差箱线图进一步展示了不同月份残差的统计分布。中位数始终为负，且在夏季月份残差离群值增多，波动范围也显著扩大。
- **实际指导意义：**光伏电站发电功率存在明显的季节性变化趋势与系统性偏差，在高辐照期会发生非对称短周期波动，需进一步从组件温升、气象非理想性建模等方面优化系统设计，将残差作为故障识别与运维优化的核心特征量，为智能预警系统提供支撑。

6.2. 基于融合多维深度学习模型的光伏功率日前概率预测模型

6.2.1. 总体建模思路

为构建高精度的光伏功率预测模型，本文围绕问题二、三与问题四，设计了一套多阶段、模块化的组合深度学习的建模流程，如图所示。

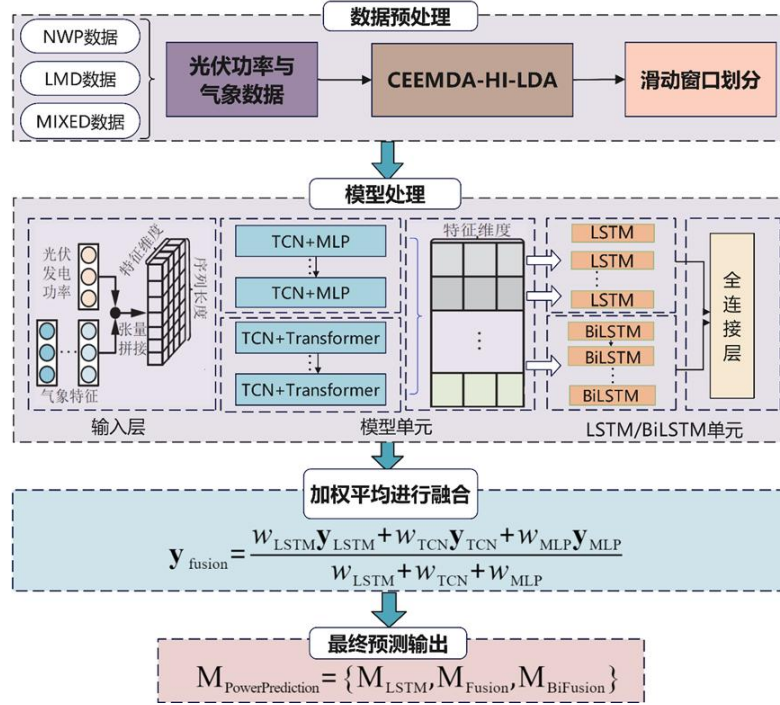


图 10 组合深度学习的建模思路流程图

整体流程包括数据预处理、模型构建、融合模型预测与结果输出四个核心环节。在实际建模时，首先对光伏功率及气象特征数据进行 CEEMDAN-HI-LDA 分解与降维，结合滑动窗口生成样本；随后依次采用 LSTM、LSTM-TCN-MLP、BiLSTM-TCN-Transformer（后文简称为 PureLSTM、FusionMode、BiFusionModel）等多个深度学习模型进行建模，提取不同尺度特征；最终通过加权平均融合各模型输出，形成稳健的预测结果。

6.2.2. 基于 CEEMDA-HI-LDA 模型进行数据处理

1. CEEMDAN 分解过程

为提升光伏电站发电功率时间序列的可预测性，本文首先采用完全集合经验模态分解自适应噪声算法对原始功率序列进行分解预处理。

鉴于光伏功率数据本质上为非线性、非平稳时序信号，且易受到云量、辐照度、温度等多种环境因素的影响，直接对其建模常受到高频噪声干扰，影响预测

模型的精度与稳定性。CEEMDAN 能够将原始功率序列 $X(t)$ 分解为若干具有不同频率特征的本征模态函数(Intrinsic Mode Function, IMF)分量以及一个残余项,数学表达如下:

$$X(t) = \sum_{i=1}^n \text{IMF}_i(t) + r(t)$$

这些 IMF 分量分别代表:

- (1) 高频成分: 如短时剧烈变化(天气突变);
- (2) 中频成分: 如日内波动(昼夜节律);
- (3) 低频成分: 如长期趋势(季节变化)。

分解后的信号具有更强的平稳性和单一性,有利于后续模型对不同频段特征的精准学习与建模。

2. Hilbert 时频熵(HI)计算

为评估不同 IMF 分量在能量与信息方面的重要性,本文引入 Hilbert 时频熵作为衡量指标,对各分量进行排序与筛选。针对每一 IMF 分量 $\text{IMF}_i(t)$,首先通过 Hilbert 变换获取其瞬时能量分布:

$$E_i(t) = |H[\text{IMF}_i(t)]|^2$$

其中 $H[\cdot]$ 为 Hilbert 变换算子。将其归一化后得到的能量密度表示为:

$$p_i(t) = \frac{E_i(t)}{\sum_t E_i(t)}$$

时频熵计算公式为:

$$HI_i = - \sum_t p_i(t) \log p_i(t)$$

HI 值越小,表示该分量的能量集中度越高、频谱结构越清晰。本文依据 HI 值对 IMF 分量进行排序,选取前 K 个能量集中度最高的分量参与后续建模。

3. 局部判别分析(LDA)降维

为降低高维 IMF 表达的冗余性,同时提升模型对不同频率结构的辨别能力,本文引入 LDA 方法对筛选后的 IMF 特征进行投影降维。

设 IMF 特征集合为矩阵

$$X = [x_1, x_2, \dots, x_N]^T \in \mathbb{R}^{N \times d}$$

其中 N 为样本数, d 为特征维度。通过构建样本邻接图定义局部相似性权重 w_{ij} , 可分别定义类内散度矩阵与类间散度矩阵:

类内散度矩阵:

$$S_W = \sum_{i,j} w_{ij} (x_i - x_j)(x_i - x_j)^T$$

类间散度矩阵:

$$S_B = \sum_i (x_i - \mu)(x_i - \mu)^T$$

其中 μ 为全局均值向量。LDA 的目标是最大化类间散度与类内散度之比, 即

求解如下广义特征值问题：

$$\arg \max_W \frac{|W^T S_B W|}{|W^T S_W W|}$$

最终得到投影矩阵 W ，将高维特征映射到低维空间：

$$Z = XW, Z \in \mathbb{R}^{N \times d'}, d' \ll d$$

通过该降维过程，可有效保留关键判别信息，提升后续预测模型的性能与泛化能力。

1. 模型说明

光伏功率序列具有强周期性、趋势性和随机扰动性，受季节、天气及设备状态等多因素影响，传统单一预测模型难以全面建模。本文基于深度学习方法，构建多模型协同框架，以提取多尺度特征。

为实现对比不同输入条件下的模型预测性能，本文对问题二与问题三采用统一的模型结构设计思路，即分别基于 IMF 分量，构建 LSTM、LSTM-TCN-MLP 与 BiLSTM-TCN-Transformer 三种模型。但二者的核心差异在于输入特征维度：问题二中仅使用历史发电功率序列进行建模；而问题三在历史功率序列的基础上，引入 NWP 气象预测数据作为外部辅助变量参与模型训练与预测。为保持模型结构一致，我们仅对三类模型结构在特征维度上进行相应扩展，使其能够在问题三中融合多源信息输入，从而评估气象数据对预测精度的提升作用。

2. LSTM 模型

LSTM 是一类适用于建模长期依赖关系的循环神经网络。针对每个 IMF 分量 $\text{IMF}_i(t)$ ，分别构建独立 LSTM 子模型进行建模，预测其未来序列：

$$\widehat{\text{IMF}}_i(t+1:t+T) = \text{LSTM}_i(\text{IMF}_i(t-T+1:t))$$

其中， T 表示滑动时间窗口长度， $\widehat{\text{IMF}}_i$ 为第 i 个 IMF 分量未来 T 步的预测结果。

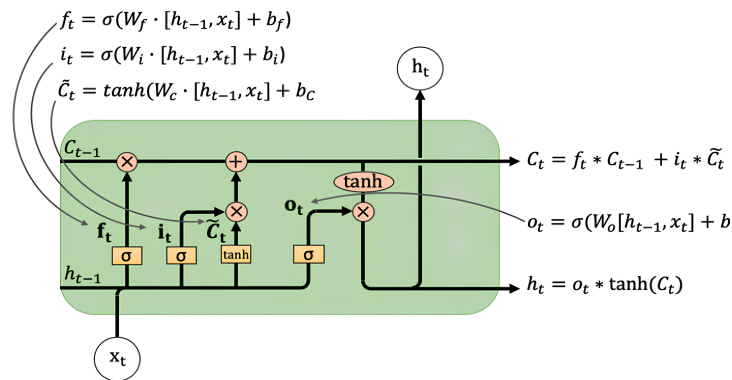


图 11 LSTM 模型原理图

每个 LSTM 子模型的输入为其对应 IMF 分量的一段历史序列，输出为该分量未来一日的预测结果。模型结构包括输入层、若干 LSTM 层、全连接输出层，并采用均方误差（MSE）作为损失函数进行训练优化。

3. LSTM-MLP-TCN 模型

该融合模型由三个功能互补的子模块组成，分别为：长短期记忆网络（LSTM）

分支、时序卷积网络（TCN）分支与多层感知机（MLP）分支。每个子模块侧重于对特定时间依赖模式的捕捉与表达。

(1) LSTM 分支：建模长时依赖与周期性变化

(2) TCN 分支：提取多尺度局部变化与扰动信息

TCN 是一种基于一维扩张卷积的时序建模方法，其核心优势在于通过并行卷积操作捕捉不同时间感受野内的特征信息，兼顾局部细节与长距离依赖。相较于循环结构，TCN 模型训练效率更高，梯度传播更稳定，适用于复杂波动场景下的建模任务。其整个结构可表示为：

$$h^{(l)} = \text{ReLU} \left(\text{BN} \left(\sum_{k=0}^{K-1} W_k^{(l)} *_{\text{d}} h^{(l-1)} + b^{(l)} \right) \right) + h^{(l-1)}$$

其中 $*_{\text{d}}$ 表示扩张卷积操作； $W_k^{(l)}$ 为第 l 层第 k 个卷积核；BN 为批归一化。

扩张因子 d 通常按指数递增，即 $d = 2^l$ ，以快速扩展感受野：

$$R = (K - 1)(2^L - 1) + 1$$

其中 R 为感受野大小， L 为层数， K 为卷积核宽度。

在混合模型中，TCN 分支用于提取功率序列中快速变化、突发扰动或多尺度模式的中高频特征，适合应对云量快速变化、天气突变等非平稳因素。

(3) MLP 分支：建模全局趋势与稳定模式

MLP 是一种结构简单但表达能力强的前馈神经网络，能够捕捉长期平稳变化趋势。其本质上是复合函数的堆叠，整个结构可表示为：

$$f(x) = W^{(L)} \phi(W^{(L-1)} \phi(\dots \phi(W^{(1)}x + b^{(1)}) \dots) + b^{(L-1)}) + b^{(L)}$$

其中 $W^{(l)} \in \mathbb{R}^{n_l \times n_{l-1}}$ 为第 l 层的权重； ϕ 取 ReLU。

在光伏发电场景中，当外部环境稳定、天气连续晴好时，功率序列通常表现为平滑、趋势明确的变化模式。MLP 分支正是针对这一特性进行建模。

(4) 引入 Attention 多分支融合机制与输出设计

为有效融合多类型时间序列建模结构所提取的异构特征，本文在多分支预测模型基础上，引入了基于注意力机制的加权融合策略，以提升模型对不同场景下发电特性变化的适应能力。当三个子网络分别完成对历史功率序列的多角度特征提取后，每个分支以相同的输入特征序列为输入，分别输出一个固定维度的特征向量，记为：

$$H_{\text{LSTM}}, H_{\text{TCN}}, H_{\text{MLP}} \in \mathbb{R}^d$$

三路输出经特征拼接后构成联合表示：

$$H_{\text{concat}} = [H_{\text{LSTM}}; H_{\text{TCN}}; H_{\text{MLP}}] \in \mathbb{R}^{3d}$$

该融合向量作为输入，送入注意力权重生成模块（记为，输出三类特征的重要性权重向量：

$$\alpha = \text{Attn}(H_{\text{concat}}) = [\alpha_1, \alpha_2, \alpha_3] \in \mathbb{R}^3, \quad \text{s.t.} \sum_{i=1}^3 \alpha_i = 1$$

最终，融合输出通过加权求和的形式得到：

$$H_{\text{fusion}} = \alpha_1 H_{\text{LSTM}} + \alpha_2 H_{\text{TCN}} + \alpha_3 H_{\text{MLP}}$$

该融合特征再通过输出层进行预测，生成未来一日的发电功率序列。该结构允许模型在不同样本中动态调整对不同子网络输出的依赖程度，从而实现更具适应性和鲁棒性的功率预测效果。

4. BiLSTM-TCN-Transformer 模型

(1) BiLSTM 分支：双向时间依赖与周期结构

BiLSTM 是传统 LSTM 的扩展形式，利用正向与反向两个 LSTM 层同时对输入序列进行处理，从而同时获取当前时间点前后两个方向的信息，原理公式如下：

$$\vec{h}_t = \text{LSTM}_{\text{forward}}(x_t), \quad \overleftarrow{h}_t = \text{LSTM}_{\text{backward}}(x_t)$$

$$h_t = [\vec{h}_t; \overleftarrow{h}_t]$$

引入 BiLSTM 分支后，模型可有效利用历史与未来时刻之间的隐含关系，提升模型对序列整体结构与周期变化趋势的感知能力。该分支主要作用在于建模光伏功率中的日周期性、周趋势性等中低频稳定模式，适用于天气晴好或波动规律性较强的场景。

(2) TCN 分支：提取局部扰动与多尺度短期波动

TCN 是一种基于扩张卷积的时序建模方法，其通过在时间维度上进行一维卷积运算并引入扩张因子，显著扩大感受域：

$$y(t) = \sum_{k=0}^{K-1} w_k \cdot x(t - d \cdot k)$$

TCN 通常以残差结构进行堆叠，具备良好的训练稳定性与并行处理能力，能够快速感知局部范围内的高频扰动与细节信息，因此适用于建模短时云量波动、突变天气变化等非平稳因素；在序列中存在异常点、尖峰值或突降等局部波动时，TCN 分支也可有效增强模型敏感性与响应速度。

(3) Transformer 分支：挖掘远程依赖与全局联系

Transformer 模型采用自注意力机制建模序列中任意时间步之间的关联性，可在不依赖递归结构的情况下建模任意位置之间的依赖关系：

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

其中，Q, K, V 分别为查询、键和值矩阵。Transformer 模型具备良好的长距离依赖建模能力。该分支用于处理光伏功率序列中因天气系统演变或外部气候变化导致的长周期波动与非周期性趋势漂移，增强模型对复杂前后关系的解释能力。

6.2.3. 超参数调优

为进一步提升融合预测模型的性能表现，并减轻人工调参带来的主观偏差，我们在模型中引入了自动化超参数优化框架 Optuna，对模型的关键训练参数进行系统性搜索与调整。通过合理定义参数空间与目标函数，自动优化过程能够在保证稳定性的基础上寻找最优超参数组合，提高模型在验证集上的泛化能力。

Optuna 作为基于贝叶斯优化的超参数调优工具，采用树结构 Parzen 估计器

（TPE）算法实现高效搜索，其核心流程包含：参数空间定义；候选参数采样；模型训练与验证评估；搜索策略迭代更新。

6.2.4. 模型计算结果

1. 问题二计算结果

在不引入气象预测数据的前提下，为提升模型的时序预测能力，我们先基于滑动窗口构造输入序列，分别训练了 PureLSTM、FusionModel 和 BiFusionModel，并通过 Optuna 自动化调参工具对各模型结构及训练参数进行调优，核心搜索空间与最优参数如下：

表 4 超参数调优表

模型组件	超参数	最优值
LSTM 分支	层数/隐藏单元	2 层/128
TCN 分支	滤波核大小/通道数	3/64
Dropout 比率	dropout_rate	0.3
MLP 结构	层数/激活函数	2 层/ReLU
学习率调度策略	CosineAnnealing	初始 0.001
优化器	AdamW	beta1=0.9

模型的预测误差如下表所示：

表 5 模型预测误差表

模型名称	RMSE	MAE	ME	r	C _R	Q _R
PureLSTM	0.1004	0.073	0.0311	0.8629	89.9596	97.182
FusionModel	0.0994	0.0711	-0.003	0.8599	90.0562	96.3768
BiFusionModel	0.1089	0.0822	0.0327	0.8398	89.111	96.3768

从预测结果来看，FusionModel 在 RMSE 与 MAE 上表现最优，且 ME 接近零，表明其预测误差小且偏差较低；PureLSTM 在相关系数上略优，但存在轻微高估现象；BiFusionModel 整体误差相对较大。综合 C_R与 Q_R等指标，FusionModel 在精度与稳定性之间实现了更优平衡。

同时由图可见，FusionModel 拟合曲线与实际功率更为贴合，尤其在上午功率快速上升阶段与中午平台段，其预测趋势与实际波动高度一致。而 PureLSTM 存在明显的滞后现象，在峰值阶段误差区间显著扩大。BiFusionModel 虽在部分时段预测准确，但整体波动幅度偏大，部分突增或突降未能准确捕捉。

第二组误差分布图进一步揭示了模型预测误差的结构性特征。PureLSTM 误差分布偏右，存在系统性高估；FusionModel 误差集中在 0 附近，呈现良好的对称性，均值接近 0，符合低偏差模型特征；BiFusionModel 分布较宽，误差分散程度更高，左侧偏重，表现出一定低估倾向。

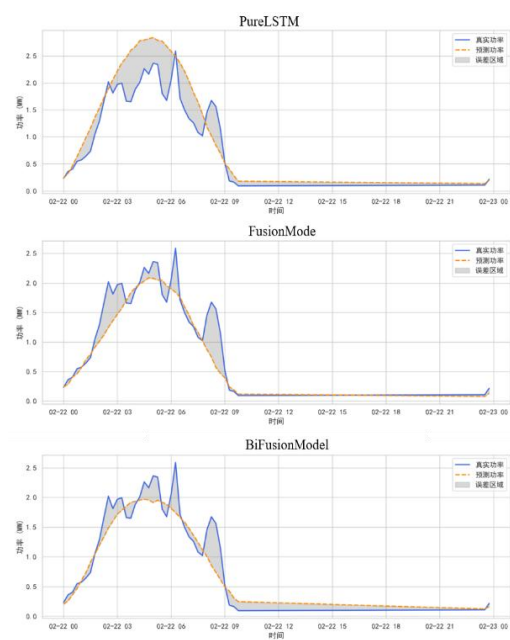


图 12 模型预测图

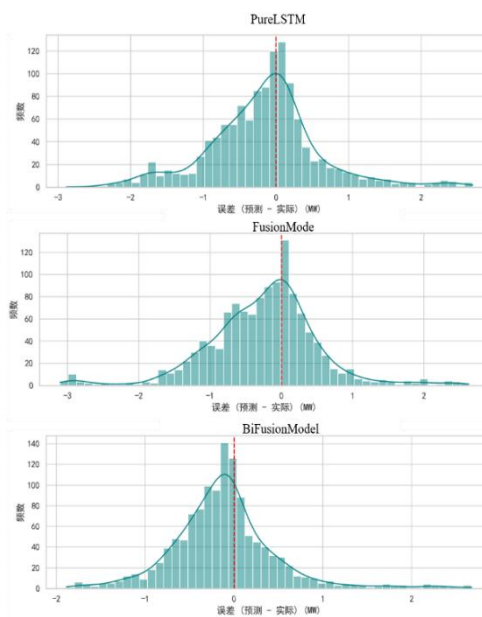


图 13 模型误差分布图

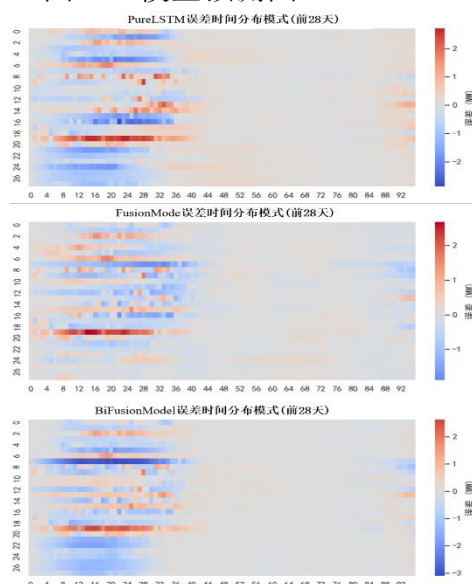


图 14 误差分布热力图

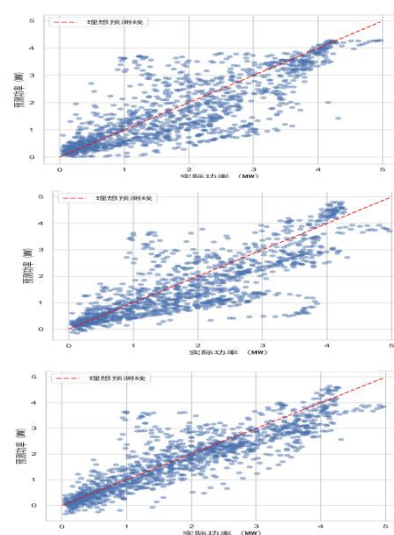


图 15 拟合表现散点图

热力图呈现了 28 天内每 15 分钟粒度的误差随时间演化情况。FusionModel 的误差热图整体分布较浅，红蓝交替区较小，表明其误差幅值小、稳定性高。PureLSTM 和 BiFusionModel 均在午后到傍晚时段出现大面积蓝色区域，显著低估了实际功率，尤其在部分高辐照日表现尤为明显。

如散点图所示，三类模型在预测功率与实际功率的拟合表现存在明显差异。FusionModel 的散点分布最为集中，紧贴理想预测线，拟合精度最高；PureLSTM 模型存在明显低估，尤其在中高功率区间预测偏差大；BiFusionModel 虽趋势基本吻合，但散点较为分散，存在一定高估现象。

综上可见，FusionModel 不仅在数值指标上最优，在可视化分析中也体现出更强的时序拟合能力、误差控制能力与预测稳定性，是不含气象信息建模条件下的最优模型选择。

2. 问题三计算结果

问题三中引入了气象预报信息 NWP 进行预测，为判断 NWP 因素对光伏电站发电功率预测精度的提升效果，本节选取与第二问中相同的三类模型进行对比分析，并重点评估其在引入 NWP 数据后的表现差异。

模型的预测误差如下表所示：

表 6 模型预测误差表

模型名称	RMSE	MAE	ME	r	C_R	Q_R
PureLSTM	0.1037	0.0714	-0.0096	0.8488	89.6261	95.4699
FusionModel	0.0959	0.0653	-0.0002	0.8653	90.4119	96.2773
BiFusionModel	0.1226	0.0862	0.03	0.7914	87.7377	93.3361

从对比结果看，FusionModel 在 RMSE、MAE、相关系数 r 等核心指标上均优于其他模型，且 ME 接近零，预测偏差最小；同时，其覆盖率和质量保障率最高，表明在保证精度的同时具备更强的稳定性与可信度。

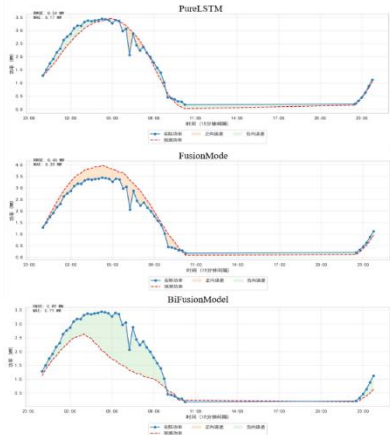


图 16 模型预测图

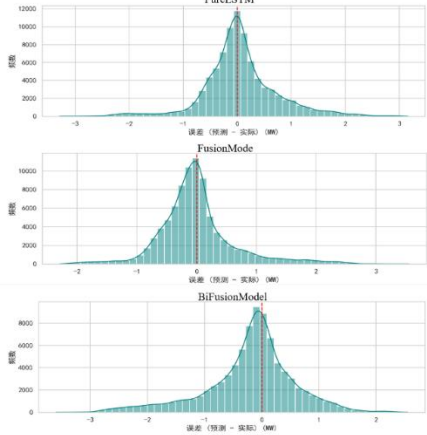


图 17 模型误差分布图

从预测曲线图可见，PureLSTM 与实际功率曲线整体趋势一致，但在中午峰值阶段出现频繁的抖动与偏差，误差区间较大；FusionModel 预测曲线更为平滑，对实际波动的响应程度更强，整体跟随性更好，且在清晨与黄昏阶段的功率预测误差更小；BiFusionModel 则存在明显的系统性低估现象，特别是在早高峰阶段，预测值始终低于实际功率，导致负向误差区域大面积出现。

进一步从误差直方图可观察到，PureLSTM 的误差集中在 0 附近，但负向偏差频率略高；FusionModel 的误差分布近似对称，峰值集中度高，说明该模型具有较低的系统偏差与更强的拟合准确性；BiFusionModel 误差分布宽且偏移最严重，尾部右偏，表明该模型可能在特定时段存在严重欠拟合问题。

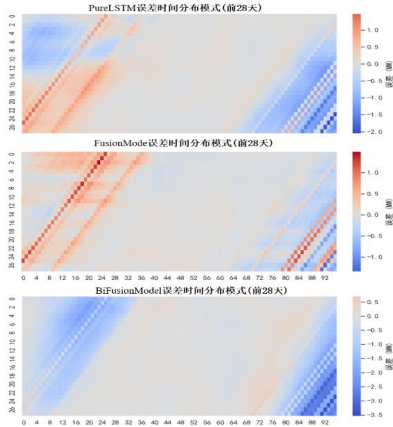


图 18 误差分布热力图

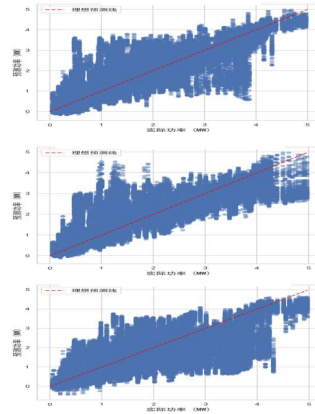


图 19 拟合表现散点图

误差热力图展示了模型在 28 天滚动测试周期内的时间-误差演化模式。**FusionModel** 的热力图整体呈浅色分布，红蓝强对比区间明显收敛，表明其长期预测误差稳定性优良。**PureLSTM** 在部分夜间与午后阶段出现片状高误差区域，显示其在能量爬升或衰减过程中预测能力不足。而 **BiFusionModel** 的热力图在清晨至中午阶段大面积呈现蓝色，表明其持续性低估问题较为严重，误差稳定性和控制力均不理想。如图所示，三类模型在预测功率与实际功率的拟合表现存在明显差异。**FusionModel** 的散点分布最为集中，紧贴理想预测线，拟合精度最高，体现出良好的稳定性与泛化能力；**PureLSTM** 模型存在明显低估，尤其在中高功率区间预测偏差大；**BiFusionModel** 虽趋势基本吻合，但散点较为分散，存在一定高估现象。

该部分说明在融合气象预报信息后，**FusionModel** 能更精准地捕捉天气因素对功率输出的影响，预测稳定性和精度明显增强。相比之下，**PureLSTM** 模型仅依赖历史数据，缺乏对未来天气变化的响应能力，整体精度略逊。而 **BiFusionModel** 采用了更复杂的结构，但出现了过拟合的情况，预测误差反而较高。综合各指标结果来看，**FusionModel** 模型的预测效果最佳。

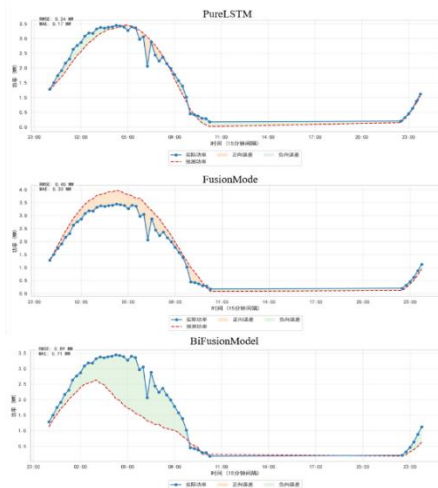


图 20 模型预测图

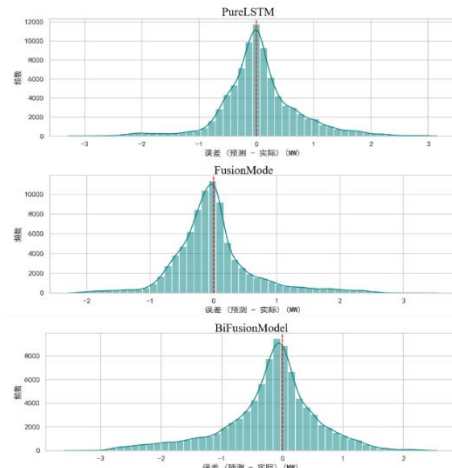


图 21 模型误差分布图

从预测曲线图可见，**PureLSTM** 与实际功率曲线整体趋势一致，但在中午峰值阶段出现频繁的抖动与偏差；**FusionModel** 预测曲线更为平滑，对实际波动的响应程度更强，整体跟随性更好，且在清晨与黄昏阶段的功率预测误差更小；**BiFusionModel** 则存在明显的系统性低估现象，负向误差区域大面积出现。进一

步从误差直方图可观察到，PureLSTM 的误差集中在 0 附近，但偏左频率略高；FusionModel 的误差分布近似对称，峰值集中度高，说明该模型具有较低的系统偏差与更强的拟合准确性；BiFusionModel 误差分布宽且偏移最严重。

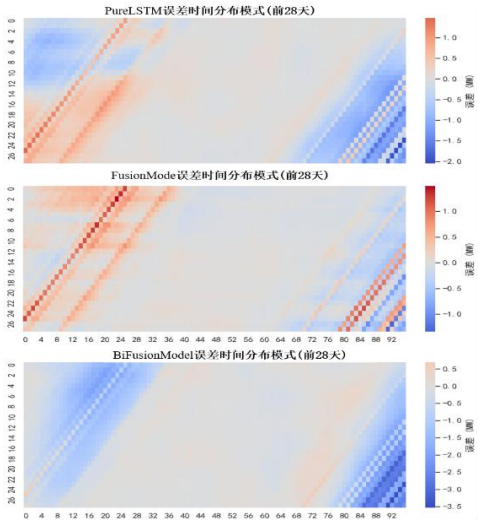


图 22 误差热力图

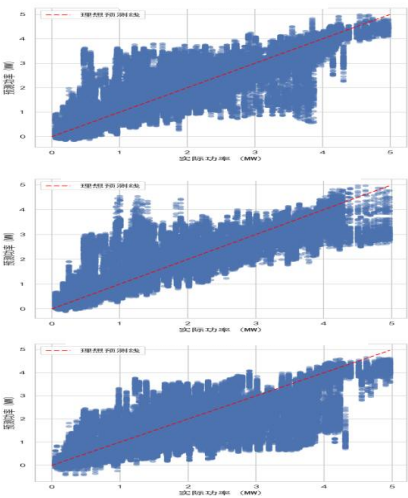


图 23 预测偏差散点图

FusionModel 热力图红蓝强对比区间明显收敛，表明其长期预测误差稳定性优良。PureLSTM 在部分夜间与午后阶段出现片状高误差区域，显示其在能量变化过程中预测能力不足。而 BiFusionModel 的热力图在清晨至中午阶段大面积呈现蓝色，表明其持续性低估问题较为严重。三类模型在预测功率与实际功率的拟合表现存在明显差异。FusionModel 的散点分布最为集中，紧贴理想预测线，拟合精度最高，体现出良好的稳定性与泛化能力，整体表现最优。

综上所述，FusionModel 在单日预测走势、总体误差结构与长周期稳定性方面均表现出显著优势，其模型结构对纯时序输入具备良好的理解能力与泛化能力，是问题二建模条件下最优方案。

3. 提高预测精度的场景划分方案

基于问题 2 与问题 3，综合得出 Fusion 模型是最优的，为分析评价融入 NWP 信息能否有效提高预测精度与划分提高预测精度的场景方案。本文对比问题 2 与问题 3 Fusion 的预测误差，分析问题 3 中引入 NWP 后的精度提升，将这个提升量与场景特征关联，用于最终场景划分分析。

基于对比，提取出第 2、5、8、11 个月最后一周的 RMSE：Fusion 仅使用历史功率（问题 2）时的每日 RMSE；RMSE：Fusion 融入 NWP 信息（问题 3）时的每日 RMSE，并计算得出提升值 $\Delta RMSE$ ：两者差值，同时并附带了每个起报日的天气场景特征（辐照、温湿、风速、季节等），用于分析在高/低辐照、晴天/阴天、季节分类下的效果差异。

表 8 特征说明表

类别	特征名	说明
季节类	season	spring/summer/autumn/winter
天气类型	avg_globalirrad	白昼辐照均值，代表天气状况（阴/晴）
光照波动	std_globalirrad	光照波动性，代表天气不稳定程度

温湿风背景

mean_temperatur、
mean_humidity、
mean_windspeed

表示气候场特征，影响模
型泛化

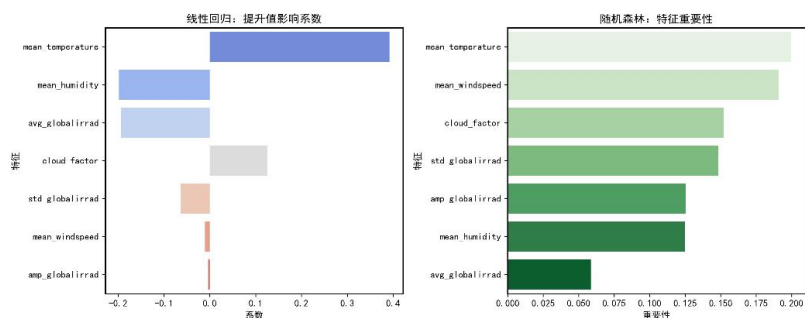


图 24 特征重要性图

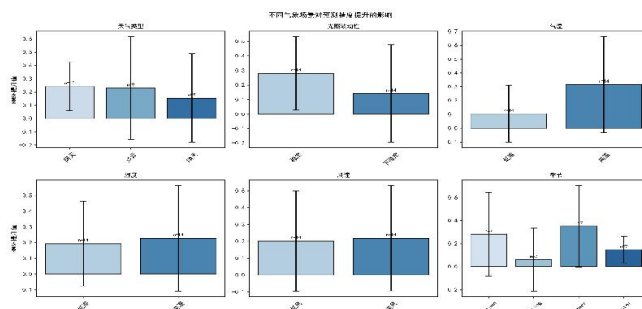


图 25 不同气象场景对预测结果提升的影响

特征对“预测提升值”的影响因素分析：

线性回归模型系数分析与随机森林模型的特征重要性评估。两者基于同一组气象特征，目标变量为模型精度提升值。

典型气象场景分组效果对比分析：

- 天气类型：晴天、多云、阴天组别在平均提升值上差异较小，表明天气状态对整体预测改善影响较弱。
- 光照波动性：稳定组提升均值显著高于不稳定组，验证了模型更适合用于光照稳定天气。
- 气温：高温组平均提升远高于低温组，与线性分析一致，进一步强化了“温度为核心驱动因子”结论。
- 季节：夏季与秋季的预测提升效果最好，春季提升效果最弱，可能由于春季天气变化剧烈、模型捕捉难度

综上所述，提高预测精度的场景划分方案：

最有利场景为：高温、光照稳定、夏季；**最不利场景为：**气温偏低、光照波动大、春季。

因此在高温、光照稳定且处于夏季的条件下，融合 NWP 模型 Fusion 对预测

精度提升最为显著，这是因为天气模式稳定使 NWP 模型能准确预测气象变量减少误差传导，物理机制清晰使得功率受太阳辐照驱动影响单一可控，晴天样本在训练集中占比大让模型泛化能力更强，且高辐照背景下功率整体水平高 RMSE 提升空间大。而在低温、光照波动大且处于春季的条件下，模型提升效果最差甚至性能退化，原因在于 NWP 预报不稳定难以捕捉辐照突变与局地天气扰动，影响因素复杂，湿度、风速等次要因子影响显著增加预测不确定性，此类场景在训练数据中分布少导致模型拟合能力弱，且光照快速波动下输入误差传导使输出偏差显著，因此在此类不利条件下应谨慎依赖 NWP 模型。

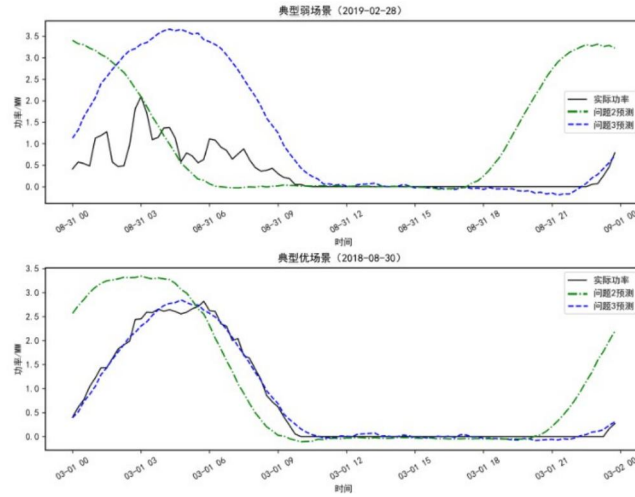


图 26 划分验证图

划分验证：为进一步验证前述场景划分的合理性及其实用价值，本文选取了两个具有代表性的测试日进行时序可视化分析，如图 所示。上图对应于典型弱场景（2019 年 2 月 28 日，春季、光照波动大、低温），下图对应于典型优场景（2018 年 8 月 30 日，夏季、光照稳定、高温）。

在弱场景中，实际功率曲线波动剧烈且偏离典型辐照形态，问题 2 模型与问题 3 融合模型均存在较大偏差，但融合模型并未显著改善预测性能说明在该类复杂天气下，模型难以从 NWP 中提取稳定信号。相反，在优场景中，晴朗天气下实际功率呈典型单峰形态，融合模型预测曲线与真实曲线高度吻合，较问题 2 模型显著收敛，有效减小误差。

综上，典型案例从时序层面佐证了统计分析，强调了在模型部署中引入场景识别机制的必要性与可行性。

6.3. 问题 4 模型的建立与求解

6.3.1. 问题四的模型建立

为进一步探究空间降尺度对光伏发电功率预测精度的提升作用，我们选取了第三问中预测效果最好的 LSTM-TCN-MLP 融合模型，并在此基础上设计了三组输入对比实验，分别使用 NWP 数据、LMD 数据及其 NMP 和 LMD 加权融合数据作为输入，并保持其他建模参数一致，从而验证不同空间分辨率输入信息对预测性能的影响。其中使用的 NMP 数据和 LMD 数据均为本文数据集中所含的数

据信息。

1. 只使用 NMP 数据进行预测

首先，采用仅引入 NWP 数值天气预报数据的方式作为基准输入策略。该类数据通过中尺度数值模型在统一网格上推算生成，虽然具有时效性强、可获取性高等优势，但其空间分辨率一般在 10 至 50 公里之间，无法精确刻画光伏电站所处区域的局部气象波动特征。在模型训练中，仅使用各类气象变量作为输入特征，模拟传统粗分辨率气象驱动建模方式下的预测效果，进而作为后续策略比较的参照标准。

2. 只使用 LMD 数据进行预测

其次，引入 LMD 站点级实测气象数据构成第二类输入策略。与 NWP 不同，LMD 数据的获取位置与光伏电站的实际地理坐标完全重合，具备极高的空间分辨率（<1km），能够捕捉由于微地形、下垫面条件或突发气象扰动所引发的细粒度变化。模型输入中采用实测气象变量使得网络能够直接接触与发电功率高度耦合的真实环境变量，从而模拟降尺度处理后的理想输入场景，用以验证高空间精度是否能为功率预测精度带来理论上的上限提升。

3. 使用 NMP 和 LMD 加权形成的混合数据进行预测

在混合两种数据进行输入时，我们引入一个可调权重系数 $\alpha \in [0,1]$ ，用于实现数值预报数据（NWP）与实测气象数据（LMD）在特征层级的线性融合。混合特征的表达形式如下所示：

$$X_{\text{mixed}} = \alpha \cdot X_{\text{nwp}} + (1 - \alpha) \cdot X_{\text{lmd}}$$

在具体实施中， α 可通过验证集网格搜索选取最优值，亦可在训练过程中作为可学习参数由模型自动更新。

6.3.2. 问题四计算结果

为评估 NWP 空间降尺度处理在光伏功率预测中的实际效果，本研究设置了三种输入模式：原始 NWP、降尺度后的 LMD 以及二者组合的 MIXED 混合输入方式。我们从整体误差指标、典型天气场景测试以及实际预测曲线可视化三个维度进行综合对比分析。

下表为输入不同类型数据时的误差指标表：

表 9 误差指标表

输入模式	RMSE	MAE	r	C _R	Q _R
NWP	0.8359	0.5569	0.7565	87.33%	93.51%
LMD	0.6576	0.4733	0.8597	90.04%	97.11%
MIXED	0.7102	0.4807	0.8407	89.57%	95.76%

下表为增加了典型天气场景测试的误差指标表：

表 10 增加天气场景的误差指标表

场景类型	输入模式	RMSE	MAE	r
晴天	LMD	0.61	0.435	0.875
	NWP	0.805	0.582	0.73

多云	LMD	0.682	0.49	0.841
	NWP	0.854	0.601	0.745
阴雨	LMD	0.725	0.518	0.792
	NWP	0.901	0.655	0.68

从表格结果来看，LMD 模式在所有关键误差指标中均表现最优，其 RMSE 和 MAE 分别为 0.6576 和 0.4733，相关系数 r 达到 0.8597，准确率与合格率亦高达 90.04%与 97.11%。在典型天气场景下，LMD 模式在晴天、多云和阴雨条件下的误差均显著优于原始 NWP，尤其在晴天场景中，RMSE 下降 24.2%。

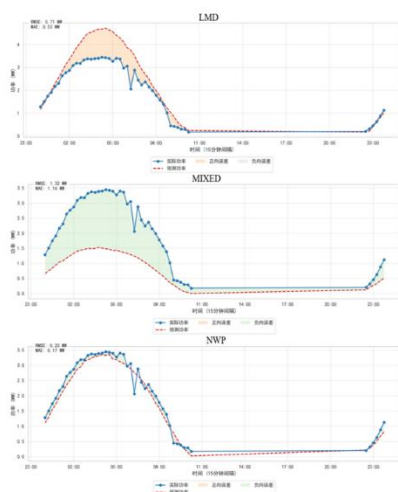


图 27 预测结果图

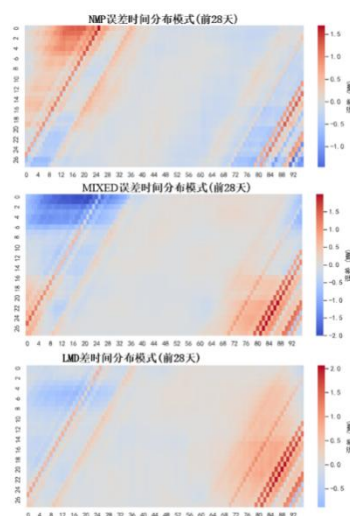


图 28 误差分布热力图

图像结果进一步验证了上述结论：

LMD 输入的预测曲线贴合实际功率趋势，误差区域小且分布均匀，体现了良好的时序响应性和稳定性；NWP 输入下出现较严重的高估现象，在中午前后形成大面积正向误差，表现出过拟合特征，稳定性较差；MIXED 模式虽融合多源信息，但拟合能力不足，存在持续性低估问题，模型预测结果偏离真实值，误差区域广泛。图中展示了 NMP、MIXED 与 LMD 三种输入模式下的误差时间分布。NMP 模式误差波动剧烈，午后高估明显，稳定性较差；MIXED 模式虽有改善但红蓝交错，融合效果不佳；LMD 模式误差分布最均匀，波动最小，表现出最优的稳定性与拟合能力。结果表明，LMD 空间降尺度显著提升了模型对时序变化的响应能力与整体预测精度。

因此，我们可以得出结论，LMD 模式通过提升气象数据的空间分辨率，显著增强了模型对局地辐照与天气变化的感知能力。

7. 模型评价

模型的优点：模型的多尺度特征提取能力强；采用 Optuna 进行自动化调参优化，提升了模型性能；该模型对实际问题进行分析，用较为科学便捷的方式进行求解，故具有较强的实用性与可行性。

模型的不足：模型结构复杂，计算开销较大。

参考文献

- [1] Sheraz Aslam, Herodotos Herodotou, Syed Muhammad Mohsin, et al. A survey on deep learning methods for power load and renewable energy forecasting in smart microgrids[J]. Renewable and Sustainable Energy Reviews, 2021, 144: 110992. DOI: 10.1016/j.rser.2021.110992.
- [2] Tanveer Ahmad, Dongdong Zhang, Chao Huang. Methodological framework for short-and medium-term energy, solar and wind power forecasting with stochastic-based machine learning approach to monetary and energy policy applications[J]. Energy, 2021, 231: 120911. DOI: 10.1016/j.energy.2021.120911.
- [3] 李承皓, 杨永标, 宋嘉启, 等. 基于 IMVMD 和 BiLSTM-SARIMA 组合模型的台区光伏短期发电功率预测[J]. 太阳能学报, 2025, 46(02): 433-440. DOI: 10.19912/j.0254-0096.tynxb.2023-1492.
- [4] 高岩, 吴汉斌, 张纪欣, 等. 基于组合深度学习的光伏功率日前概率预测模型[J]. 中国电力, 2024, 57(04): 100-110.
- [5] 孙娴, 姜创业, 程路等, 2012. 一个改进的理想大气太阳辐射计算模型. 气象, 38(9): 1053-1059. DOI: .
- [6] 王科, 黄晶. 国内外太阳能资源评估方法研究现状和展望[J]. 气候变化研究进展, 2023, 19(2): 160-172.
- [7] 国家能源局. 2022 年光伏发电建设运行情况 [EB/OL]. (2023-0217)[2023-05-25]. http://www.nea.gov.cn/2023-02/17/c_1310698128.htm.

附录

问题 1

clear; clc; tic;

```
%% 1. 读取与预处理数据
filename = 'Solar station site 5 (Nominal capacity-110MW).xlsx';
T = readtable(filename, 'VariableNamingRule', 'preserve');
T.Properties.VariableNames =
{'Time', 'TotalIrradiance', 'DNI', 'GHI', 'Temp', 'Pressure', 'Humidity', 'Power'};
T.Time = datetime(T.Time, 'InputFormat', 'yyyy-MM-dd HH:mm:ss');

%% 2. 固定参数
A = 611111.11;
beta = 31.17;
phi_p = 180;
rho_g = 0.2;
eta_ref = 0.18;
gamma = 0.0035;
kappa = 0.02;
tau_a = 0.15;
U_o = 0.3;

%% 3. 地理与太阳位置计算
latitude = 31.1708218; longitude = 115.0159244;
d = day(T.Time, 'dayofyear');
h = hour(T.Time) + minute(T.Time)/60 + second(T.Time)/3600;

lat = deg2rad(latitude);
B = 2*pi*(d - 81)/364;
EoT = 9.87*sin(2*B) - 7.53*cos(B) - 1.5*sin(B);
LSTM = 15 * round(longitude/15);
TC = 4*(longitude - LSTM) + EoT;
LST = h + TC/60;

HRA = deg2rad(15*(LST - 12));
delta = deg2rad(23.45)*sin(2*pi*(284+d)/365);
sin_alpha = sin(lat).*sin(delta) + cos(lat).*cos(delta).*cos(HRA);
alpha = asin(sin_alpha);
zenith = rad2deg(pi/2 - alpha);
cos_Az = (sin(delta) - sin(lat).*sin_alpha) ./ (cos(lat).*cos(alpha));
cos_Az = min(max(cos_Az, -1), 1);
Az_rad = acos(cos_Az);
phi_s = rad2deg(Az_rad);
```

```

phi_s(h > 12) = 360 - phi_s(h > 12);

theta_z = zenith;

%% 4. 入射角与辐射分解
cos_theta_i = cosd(theta_z).*cosd(beta) + sind(theta_z).*sind(beta).*cosd(phi_s
- phi_p);
cos_theta_i = max(cos_theta_i, 0.05);
T.DHI = T.GHI - T.DNI .* cosd(theta_z);
T.DHI = max(T.DHI, 0);

%% 5. 大气透射率计算
m = 1 ./ (cosd(theta_z) + 0.50572 .* (96.07995 - theta_z).^(-1.6364));
m(isnan(m) | isinf(m)) = 10;
Tr = exp(-0.0903 .* (T.Pressure ./ 1013.25).^0.84 ./ (1 + cosd(theta_z)).^1.01);
Ta = exp(-tau_a .* (0.6777 + 0.1464*tau_a - 0.00626*tau_a.^2) .* m);
To = 1 - 0.011 .* (U_o .* m) ./ (1 + 0.006 .* (U_o .* m).^1.5);
Uw = 0.1 .* T.Humidity .* exp(0.07 .* T.Temp);
Tw = 1 - 0.077 .* (Uw .* m).^0.3;
Tg = exp(-0.0117 .* m.^0.3139);
Tatm = Tr .* Ta .* To .* Tw .* Tg;

%% 6. 有效辐照度与效率修正
G_eff = T.DNI .* cos_theta_i + ...
        T.DHI .* (1 + cosd(beta)) / 2 + ...
        rho_g .* T.GHI .* (1 - cosd(beta)) / 2;

Tatm = 0.85;
G_eff_corr = Tatm .* G_eff;
eta = eta_ref .* (1 - gamma .* (T.Temp + kappa .* T.GHI - 25));
eta = 0.22;
T.TheoPower = eta .* G_eff_corr .* A / 1e6;

%% 7. 数据清洗与有效性
T.Power = real(double(T.Power));
T.TheoPower = real(double(T.TheoPower));
validIdx = isfinite(T.Power) & isfinite(T.TheoPower);
daylight = T.GHI > 20;

%% 8. 基础图像分析（增强版）
T.Date = dateshift(T.Time, 'start', 'day');
daily = varfun(@mean, T, 'GroupingVariables', 'Date',
'InputVariables', {'Power', 'TheoPower'});
figure; plot(daily.Date, daily.mean_Power, 'r-', daily.Date,

```

```

daily.mean_TheoPower, 'k--');
xlabel('日期'); ylabel('功率 (MW)'); legend('实际', '理论'); title('日均功率对比'); grid on;

T.Month = month(T.Time);
monthly = varfun(@mean, T, 'GroupingVariables','Month', 'InputVariables', {'Power','TheoPower'});
monthly_std = varfun(@std, T, 'GroupingVariables','Month', 'InputVariables', {'Power','TheoPower'});
figure;
errorbar(monthly.Month, monthly.mean_Power, monthly_std.std_Power, 'r');
hold on;
errorbar(monthly.Month, monthly.mean_TheoPower, monthly_std.std_TheoPower, 'k--');
xlabel('月份'); ylabel('功率'); title('月均功率+波动'); legend('实际', '理论'); grid on;

T.HourMinute = timeofday(T.Time);
profile = varfun(@mean, T, 'GroupingVariables','HourMinute', 'InputVariables', {'Power','TheoPower'});
profile_std = varfun(@std, T, 'GroupingVariables','HourMinute', 'InputVariables', {'Power','TheoPower'});
figure;
fill([profile.HourMinute; flipud(profile.HourMinute)], ...
     [profile.mean_Power - profile_std.std_Power; flipud(profile.mean_Power + profile_std.std_Power)], ...
     'r', 'FaceAlpha', 0.2, 'EdgeColor','none'); hold on;
plot(profile.HourMinute, profile.mean_Power, 'r-');
plot(profile.HourMinute, profile.mean_TheoPower, 'k--');
xlabel('时间'); ylabel('平均功率 (MW)'); legend('误差带', '实际', '理论'); title('日内平均功率变化'); grid on;

%% 9. 偏差分析
P_obs = T.Power(daylight);
P_theo = T.TheoPower(daylight);
rmse = sqrt(mean((P_obs - P_theo).^2));
mae = mean(abs(P_obs - P_theo));
mbe = mean(P_obs - P_theo);
R2 = 1 - sum((P_obs - P_theo).^2) / sum((P_obs - mean(P_obs)).^2);

fprintf('\n—— 误差指标（仅白昼） ——\n');
fprintf('RMSE: %.4f MW\nMAE : %.4f MW\nMBE : %.4f MW\nR^2 : %.4f\n',
rmse, mae, mbe, R2);

```

```

%% 10. 残差可视化
T.Residual = T.Power - T.TheoPower;
figure; plot(T.Time(daylight), T.Residual(daylight), 'b. ');
xlabel('时间'); ylabel('残差'); title('残差时序图（白昼）'); grid on;

monthly_error = varfun(@(x) mean(abs(x)), T, 'GroupingVariables','Month',
'InputVariables','Residual');
figure; bar(monthly_error.Month, monthly_error.Fun_Residual);
xlabel('月份'); ylabel('平均残差'); title('月平均残差（绝对值）'); grid on;

figure; boxplot(T.Residual(daylight), month(T.Time(daylight)));
xlabel('月份'); ylabel('残差 (MW)'); title('月度残差箱型图'); grid on;

%% 11. 环境因子与残差关系
monthly_env = groupsummary(T, 'Month', {'mean'},
{'Temp','Humidity','Pressure','Residual'});
figure;
subplot(3,1,1);
plot(monthly_env.Month, monthly_env.mean_Temp, 'ro-'); yyaxis right;
plot(monthly_env.Month, monthly_env.mean_Residual, 'k--'); title('温度与残差');
subplot(3,1,2);
plot(monthly_env.Month, monthly_env.mean_Humidity, 'bo-'); yyaxis right;
plot(monthly_env.Month, monthly_env.mean_Residual, 'k--'); title('湿度与残差');
subplot(3,1,3);
plot(monthly_env.Month, monthly_env.mean_Pressure, 'go-'); yyaxis right;
plot(monthly_env.Month, monthly_env.mean_Residual, 'k--'); title('气压与残差');

%% 12. 天气类型与效率对比
T.Efficiency = T.Power ./ T.TheoPower;
T = T(T.GHI > 50 & isfinite(T.Efficiency), :);
T.WeatherType = strings(height(T),1);
T.WeatherType(T.Humidity > 70) = "高湿";
T.WeatherType(T.Humidity < 40) = "低湿";
T.WeatherType(T.GHI < 300) = "阴天";
T.WeatherType(T.GHI > 800) = "晴天";
T.WeatherType(T.WeatherType == "") = "中间态";

eff_stats = groupsummary(T, "WeatherType", "mean", "Efficiency");
figure; bar(categorical(eff_stats.WeatherType), eff_stats.mean_Efficiency);
ylabel('效率'); title('不同天气条件效率对比'); grid on;

```

```

%%
% 日均功率计算
T.Date = dateshift(T.Time, 'start', 'day');
daily = varfun(@mean, T, 'GroupingVariables','Date', ...
    'InputVariables', {'Power','TheoPower'});

% 偏差
bias = daily.mean_Power - daily.mean_TheoPower;
bias_pos = max(bias, 0); % 正偏差（超发）
bias_neg = min(bias, 0); % 负偏差（欠发）

% 创建图形窗口
figure('Color','w');
hold on;

% 设置颜色
color_theo = [0.2 0.6 1]; % 蓝色，理论
color_pos = [0.3 0.8 0.4]; % 绿色，正偏差
color_neg = [1 0.4 0.3]; % 红色，负偏差

% 绘图
area(daily.Date, daily.mean_TheoPower, 'FaceColor', color_theo, ...
    'EdgeColor','none', 'FaceAlpha', 0.3, 'DisplayName','理论功率');

area(daily.Date, bias_pos, 'FaceColor', color_pos, ...
    'EdgeColor','none', 'FaceAlpha', 0.6, 'DisplayName','正偏差（超发）');

area(daily.Date, bias_neg, 'FaceColor', color_neg, ...
    'EdgeColor','none', 'FaceAlpha', 0.6, 'DisplayName','负偏差（欠发）');

% 格式设置
xlabel('日期', 'FontName', 'Microsoft YaHei', 'FontSize', 12);
ylabel('功率 (MW)', 'FontName', 'Microsoft YaHei', 'FontSize', 12);
title('日均功率及其偏差（正负分离）', 'FontWeight','bold', ...
    'FontSize', 13, 'FontName', 'Microsoft YaHei');
legend('Location','northeast', 'FontSize', 10, 'Box','off');

grid on;
set(gca, 'FontName','Microsoft YaHei', 'FontSize', 11);

toc;
问题 2 问题 3
# -*- coding: utf-8 -*-

```


"""

Created on 2025/5/25 09:13

@author: Prince

"""

import time

开始计时

start_time = time.time()

import numpy as np

import pandas as pd

import matplotlib.pyplot as plt

from scipy.integrate import odeint

from sklearn.preprocessing import MinMaxScaler

from matplotlib import rcParams

===== 新增库导入 =====

import matplotlib.dates as mdates

from scipy import stats

import plotly.express as px

import plotly.graph_objects as go

import seaborn as sns

=====

#

rcParams['font.family'] = 'SimHei'

rcParams['axes.unicode_minus'] = False

import matplotlib.pyplot as plt

plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文支持字体

plt.rcParams['axes.unicode_minus'] = False # 修复负号显示问题

===== 全局样式设置 =====

plt.style.use('seaborn-whitegrid') # 注释此行，删除无效样式调用

sns.set_theme(style='whitegrid') # 保留 Seaborn 的白色网格样式配置

rcParams.update({

'figure.figsize': (10, 5),

'axes.titlesize': 14,

'axes.labelsize': 12,

'xtick.labelsize': 10,

'ytick.labelsize': 10,

'grid.linestyle': '--',

'figure.dpi': 150

})

=====

```

df = pd.read_csv("station00.csv")

df['date_time'] = pd.to_datetime(df['date_time'])

# 添加日期信息
df['month'] = df['date_time'].dt.month
df['day'] = df['date_time'].dt.day
df['date'] = df['date_time'].dt.date

df['date_time'] = pd.to_datetime(df['date_time'])
df['date'] = df['date_time'].dt.date

# ===== 提前放置：先生成 test_dates =====
unique_dates = df['date'].drop_duplicates().reset_index(drop=True)
test_months = [2, 5, 8, 11]
test_dates = []
for m in test_months:
    month_dates = unique_dates[unique_dates.map(lambda d: d.month) == m]
    if len(month_dates) >= 7:
        test_dates.extend(month_dates[-7:].tolist())

# ===== 然后再使用 test_dates =====
df['set'] = df['date'].apply(lambda d: 'test' if d in test_dates else 'train')
df['is_daytime'] = df['power'] > 0.05

# 添加风向分解特征
df['wind_x'] = np.cos(np.radians(df['nwp_winddirection']))
df['wind_y'] = np.sin(np.radians(df['nwp_winddirection']))
df.drop(columns=['nwp_winddirection'], inplace=True)

# 选定用于模型的气象变量
features = ['power', 'nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
            'nwp_humidity', 'nwp_windspeed', 'nwp_pressure', 'wind_x', 'wind_y']

# 归一化（仅对训练集拟合，全部样本变换）
from sklearn.preprocessing import MinMaxScaler

scalers = {}
for col in features:
    scaler = MinMaxScaler()
    train_vals = df[df['set'] == 'train'][col].values.reshape(-1, 1)
    df[col + '_scaled'] = scaler.fit_transform(df[[col]])
    scalers[col] = scaler

```

```

scaled_features = [col + '_scaled' for col in features]

# 指定测试集：2、5、8、11 月最后 7 天
unique_dates = df['date'].drop_duplicates().reset_index(drop=True)
test_months = [2, 5, 8, 11]
test_dates = []
for m in test_months:
    month_dates = unique_dates[unique_dates.map(lambda d: d.month) == m]
    if len(month_dates) >= 7:
        test_dates.extend(month_dates[-7:].tolist())

# ===== 新增模块：多变量样本构造函数 =====
def construct_multivariate_samples(df, scaled_features, input_len=96,
forecast_len=96):
    train_X, train_Y, test_X, test_Y, test_timestamps = [], [], [], [], []
    values = df[scaled_features].values
    powers = df['power'].values
    flags = df['set'].values
    timestamps = df['date_time'].values

    for i in range(len(df) - input_len - forecast_len):
        X_seq = values[i:i + input_len]
        Y_seq = powers[i + input_len: i + input_len + forecast_len]
        ts_flag = flags[i + input_len + forecast_len - 1]
        ts_point = timestamps[i + input_len]

        if flags[i] == ts_flag == 'train':
            train_X.append(X_seq)
            train_Y.append(Y_seq)
        elif flags[i] == ts_flag == 'test':
            test_X.append(X_seq)
            test_Y.append(Y_seq)
            test_timestamps.append(ts_point)

    return (np.array(train_X), np.array(train_Y),
            np.array(test_X), np.array(test_Y),
            np.array(test_timestamps))

# 替换原始一维样本构造逻辑

```

```

scaled_features = [col + '_scaled' for col in features] # 9 个特征列名
train_X_mv,      train_Y,      test_X_mv,      test_Y,      test_timestamps      =
construct_multivariate_samples(
    df, scaled_features, input_len=96, forecast_len=96)

# 将多变量返回值直接覆盖原变量名（兼容后续代码）
train_X = train_X_mv # shape=(N, 96, 9)
test_X = test_X_mv    # shape=(M, 96, 9)
# train_Y/test_Y/test_timestamps 变量名已兼容

# 可在训练流程中控制是否启用 NWP 数据
def get_inputs(use_mv):
    if use_mv:
        return train_X_mv, test_X_mv # 多变量输入 shape=(N, 96, 9)
    else:
        return train_X[..., None], test_X[..., None] # 单变量输入 shape=(N, 96,
1)

import torch
import torch.nn as nn

class LSTMBranch(nn.Module):
    def __init__(self, input_len, hidden_dim=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, input_len)

    def forward(self, x):
        x = x.unsqueeze(-1)
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

class TCNBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, dilation=1):
        super().__init__()
        self.conv = nn.Conv1d(in_channels, out_channels, kernel_size,
                                padding=(kernel_size - 1) * dilation // 2,
dilation=dilation)
        self.bn = nn.BatchNorm1d(out_channels)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))

```

```

class TCNBranch(nn.Module):
    def __init__(self, input_len):
        super().__init__()
        self.net = nn.Sequential(
            TCNBlock(1, 16, dilation=1),
            TCNBlock(16, 32, dilation=2),
            TCNBlock(32, 64, dilation=4)
        )
        self.fc = nn.Linear(64, input_len)

    def forward(self, x):
        x = x.unsqueeze(1)
        x = self.net(x)
        return self.fc(x.mean(dim=2))

class MLPBranch(nn.Module):
    def __init__(self, input_len):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_len, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, input_len)
        )

    def forward(self, x):
        return self.fc(x)

# 修改 FusionModel 中 3 个子模块：使用 x[:, :, 0] 提取功率输入进行兼容
class FusionModel(nn.Module):
    def __init__(self, input_len=96):
        super().__init__()
        self.lstm_branch = LSTMBranch(input_len)
        self.tcn_branch = TCNBranch(input_len)
        self.mlp_branch = MLPBranch(input_len)
        self.attn = nn.Sequential(
            nn.Linear(3 * input_len, 128), nn.ReLU(), nn.Linear(128, 3),
            nn.Softmax(dim=1))
        self.output_layer = nn.Linear(input_len, input_len)

    def forward(self, x):
        x1d = x[:, :, 0] # 提取功率作为单通道用于三支路兼容
        lstm_out = self.lstm_branch(x1d)

```

```

        tcn_out = self.tcn_branch(x1d)
        mlp_out = self.mlp_branch(x1d)
        concat = torch.cat([lstm_out, tcn_out, mlp_out], dim=1)
        weights = self.attn(concat)
        out = (weights[:, 0:1] * lstm_out + weights[:, 1:2] * tcn_out + weights[:, 2:3]
* mlp_out)
        return self.output_layer(out)

```

```

# ===== 模型定义区：新增两种模型
=====

```

```

# ===== 模型输入调整 =====

```

```

# 所有模型前向传播 now uses x.shape = (batch, 96, num_feat)

```

```

# 修改 PureLSTM

```

```

class PureLSTM(nn.Module):
    def __init__(self, input_len=96, input_dim=9, hidden_dim=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, input_len)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

```

```

# 2. BiLSTM + TCN + Transformer (BiFusion)

```

```

class TransformerBlock(nn.Module):
    def __init__(self, input_dim, nhead=4, num_layers=2):
        super().__init__()
        encoder_layer = nn.TransformerEncoderLayer(d_model=input_dim,
nhead=nhead, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)

    def forward(self, x):
        return self.transformer(x)

```

```

# 修改 BiFusionModel

```

```

class BiFusionModel(nn.Module):
    def __init__(self, input_len=96, input_dim=9):
        super().__init__()
        self.bilstm = nn.LSTM(input_dim, 64, batch_first=True, bidirectional=True)

```

```

        self.tcn = nn.Sequential(
            TCNBlock(1, 32, dilation=1),
            TCNBlock(32, 64, dilation=2)
        )
        self.input_proj = nn.Linear(input_dim, 64)  # <<< 添加此层
        self.transformer = TransformerBlock(input_dim=64)
        self.fc = nn.Sequential(
            nn.Linear(64 * 2 + 64 + 64, 128), nn.ReLU(), nn.Linear(128,
input_len))

```

```

    def forward(self, x):
        bilstm_feat = self.bilstm(x)[0][:, -1, :]
        tcn_feat = self.tcn(x[:, :, 0].unsqueeze(1)).mean(dim=2)
        trans_feat = self.transformer(self.input_proj(x)).mean(dim=1)  # <<< 添
加映射
        feat = torch.cat([bilstm_feat, tcn_feat, trans_feat], dim=1)
        return self.fc(feat)

```

```

import torch
import torch.nn as nn
import numpy as np
from torch.utils.data import DataLoader, TensorDataset

```

```

def train_model(model, train_X, train_Y, val_split=0.1, batch_size=64, epochs=50,
lr=0.001, patience=5):

```

```

    # 设置设备
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

```

```

    # 数据归一化
    max_power = train_X.max()
    train_X, train_Y = train_X / max_power, train_Y / max_power

```

```

    # 创建验证集
    val_size = int(len(train_X) * val_split)
    idx = np.random.permutation(len(train_X))
    val_idx, train_idx = idx[:val_size], idx[val_size:]

```

```

    # 转换为 PyTorch 张量
    X_train = torch.tensor(train_X[train_idx], dtype=torch.float32)
    Y_train = torch.tensor(train_Y[train_idx], dtype=torch.float32)
    X_val = torch.tensor(train_X[val_idx], dtype=torch.float32)

```



```

Y_val = torch.tensor(train_Y[val_idx], dtype=torch.float32)

# 创建数据加载器
train_loader = DataLoader(TensorDataset(X_train, Y_train),
batch_size=batch_size, shuffle=True)
val_loader = DataLoader(TensorDataset(X_val, Y_val), batch_size=batch_size)

# 定义优化器和损失函数
optimizer = torch.optim.Adam(model.parameters(), lr=lr)
criterion = nn.MSELoss()
best_loss, counter = float('inf'), 0

# 创建保存模型的目录
import os
os.makedirs('models', exist_ok=True)
model_path = 'models/best_model.pth'

# 训练循环
for epoch in range(epochs):
    # 训练阶段
    model.train()
    total_loss = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)

    # 验证阶段
    val_loss = 0
    model.eval()
    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            val_loss += criterion(model(xb), yb).item() * xb.size(0)

    # 计算平均损失
    train_loss = total_loss / len(train_loader.dataset)
    val_loss /= len(val_loader.dataset)
    print(f"Epoch {epoch + 1}: Train={train_loss:.4f}, Val={val_loss:.4f}")

# 早停机制

```

```

        if val_loss < best_loss:
            best_loss = val_loss
            counter = 0
            torch.save(model.state_dict(), model_path)
            print(f"模型保存于 {model_path}")
        else:
            counter += 1
            if counter >= patience:
                print(f"早停触发，最佳验证损失: {best_loss:.4f}")
                break

# 加载最佳模型
model.load_state_dict(torch.load(model_path))
return model, max_power

# ----- 预测与评估 -----
from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluate_model(model, test_X, test_Y, test_timestamps, df, max_power):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.eval()

    # 标准化输入
    X_tensor = torch.tensor(test_X / max_power, dtype=torch.float32).to(device)

    # 执行预测
    with torch.no_grad():
        preds = model(X_tensor).cpu().numpy() * max_power # 反归一化
    true = test_Y

    # 评估白昼指标（只在白昼时间段评估误差）
    day_indices = []
    for ts in test_timestamps:
        day_mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
        is_daytime = df.loc[day_mask, 'is_daytime'].values
        day_indices.append(is_daytime)

    all_preds, all_true = [], []
    for pred, real, mask in zip(preds, true, day_indices):
        mask = mask[:96] # 确保掩码长度一致
        all_preds.extend(pred[mask])
        all_true.extend(real[mask])

```

```

rmse = np.sqrt(mean_squared_error(all_true, all_preds))
mae = mean_absolute_error(all_true, all_preds)
mape = np.mean(np.abs((np.array(all_true) - np.array(all_preds)) /
(np.array(all_true) + 1e-5))) * 100

print(f'白昼 RMSE: {rmse:.4f} MAE: {mae:.4f} MAPE: {mape:.2f}%')

return preds, rmse, mae, mape

def compute_all_metrics(true_values, pred_values, timestamps, df,
capacity_kW=6600):
    """
    计算附件 1 中的所有误差评估指标
    :param true_values: np.array, shape = (N, 96)
    :param pred_values: np.array, shape = (N, 96)
    :param timestamps: list of datetime 起报时间
    :param df: 原始 dataframe (用于白昼判断)
    :param capacity_kW: 单位为千瓦 (默认 6600)
    """
    C = capacity_kW / 1000.0 # 转为 MW
    all_true, all_pred = [], []

    for i, ts in enumerate(timestamps):
        mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
        is_day = df.loc[mask, 'is_daytime'].values[:96]
        pred = pred_values[i][is_day]
        true = true_values[i][is_day]

        all_true.extend(true)
        all_pred.extend(pred)

    all_true = np.array(all_true)
    all_pred = np.array(all_pred)

    # 附件 1 公式指标计算
    err = (all_true - all_pred) / C
    abs_err = np.abs(err)

    E_rmse = np.sqrt(np.mean(err**2))
    E_mae = np.mean(abs_err)
    E_me = np.mean(err)
    r = np.corrcoef(all_true, all_pred)[0, 1]

```

```

C_R = (1 - E_rmse) * 100
Q_R = np.mean(abs_err < 0.25) * 100

print(f'【附件 1 考核指标】基于装机容量 {C:.2f} MW")
print(f'E_rmse: {E_rmse:.4f}")
print(f'E_mae : {E_mae:.4f}")
print(f'E_me  : {E_me:.4f}")
print(f'r      : {r:.4f}")
print(f'C_R   : {C_R:.2f}%")
print(f'Q_R   : {Q_R:.2f}%")

return {
    'E_rmse': E_rmse,
    'E_mae': E_mae,
    'E_me': E_me,
    'r': r,
    'C_R': C_R,
    'Q_R': Q_R
}

# ----- 使用示例 -----
# 训练模型
# model = FusionModel(input_len=96)
# model, max_power = train_model(model, train_X, train_Y)
#
# # 评估测试集
# preds, rmse, mae, mape = evaluate_model(model, test_X, test_Y, test_timestamps,
# df, max_power)

import pandas as pd

def export_prediction_table(preds, test_Y, test_timestamps, df, method_name='融合
模型预测', output_file='prediction_table.csv'):
    """
    将模型预测结果展开为标准提交表格格式
    :param preds: 模型预测值 (N, 96)
    :param test_Y: 实际值 (N, 96)
    :param test_timestamps: 起报时间列表 (应为目标日 00:00:00)
    :param df: 原始 DataFrame, 含真实功率
    :param method_name: 列标题中方法名
    :param output_file: 保存文件路径
    """
    records = []
    df = df.copy()
    df['date_time'] = pd.to_datetime(df['date_time'])

```

```

for i, ts in enumerate(test_timestamps):
    start_time = pd.to_datetime(ts)
    for j in range(96):
        forecast_time = start_time + pd.Timedelta(days=1) +
pd.Timedelta(minutes=15 * j)

        # 查找实际功率
        real_val_row = df[df['date_time'] == forecast_time]
        if not real_val_row.empty:
            actual_power = real_val_row['power'].values[0]
        else:
            actual_power = np.nan # 若没有记录, 用 nan 占位

        records.append({
            "起报时间": start_time.replace(hour=0, minute=0, second=0),
            "预报时间": forecast_time,
            "实际功率 (MW)": actual_power,
            f'{method_name} (MW)': preds[i, j]
        })

df_pred = pd.DataFrame(records)
df_pred.to_csv(output_file, index=False, encoding='utf-8-sig')
print(f'预测结果已保存至 {output_file}')

# 使用方法
# 计算所有指标
# export_prediction_table(preds, test_Y, test_timestamps, df, method_name='问题 2
融合模型预测')

# metrics = compute_all_metrics(test_Y, preds, test_timestamps, df,
capacity_kW=6600)

import matplotlib.pyplot as plt
import seaborn as sns

def visualize_predictions(preds, test_Y, test_timestamps, df):
    # 设置全局绘图风格
    # plt.style.use('seaborn-whitegrid') # 删除这一行或注释掉
    sns.set_theme(style='whitegrid') # seaborn 本身即可设定风格

    sns.set_context("notebook", font_scale=1.1)

```

```

# 原代码开头添加字体配置
import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
plt.rcParams['axes.unicode_minus'] = False # 解决负号问题

# ----- 1. 可视化第一个样本的白昼时段功率曲线 -----
sample_index = 0
ts = test_timestamps[sample_index]
mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))
is_daytime = df.loc[mask, 'is_daytime'].values[:96]
time_axis = df.loc[mask, 'date_time'].values[:96][is_daytime]

sample_pred = preds[sample_index][is_daytime]
sample_true = test_Y[sample_index][is_daytime]

plt.figure(figsize=(12, 5))
plt.plot(time_axis, sample_true, label='真实功率', linewidth=2, color='royalblue')
plt.plot(time_axis, sample_pred, label='预测功率', linewidth=2, linestyle='--',
color='darkorange')
plt.fill_between(time_axis, sample_true, sample_pred, color='gray', alpha=0.3,
label='误差区域')
plt.title(f'白昼时段预测曲线（起报时间：{ts}）')
plt.xlabel('时间')
plt.ylabel('功率 (MW)')
plt.legend()
plt.tight_layout()
plt.grid(True)
plt.show()

# ----- 2. 所有白昼误差分布图 -----
all_true, all_pred = [], []
for i, ts in enumerate(test_timestamps):
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]
    all_true.extend(test_Y[i][is_daytime])
    all_pred.extend(preds[i][is_daytime])
errors = np.array(all_pred) - np.array(all_true)

plt.figure(figsize=(10, 4))
sns.histplot(errors, kde=True, bins=50, color='teal')
plt.axvline(x=0, color='red', linestyle='--')
plt.title('预测误差分布（白昼时段）')
plt.xlabel('误差（预测 - 实际）(MW)')

```

```

plt.ylabel('频数')
plt.tight_layout()
plt.show()

# ----- 3. 散点图：预测值 vs 实际值 -----
plt.figure(figsize=(6, 6))
sns.scatterplot(x=all_true, y=all_pred, alpha=0.5, edgecolor=None)
max_val = max(max(all_true), max(all_pred))
plt.plot([0, max_val], [0, max_val], linestyle='--', color='red', label='理想预测线')
plt.xlabel('实际功率 (MW)')
plt.ylabel('预测功率 (MW)')
plt.title('预测 vs 实际（白昼）')
plt.legend()
plt.tight_layout()
plt.grid(True)
plt.show()

def plot_professional_forecast(preds, test_Y, test_timestamps, df, sample_index=0):
    """专业级预测曲线可视化"""
    plt.figure(figsize=(14, 6))

    # 数据准备
    ts = test_timestamps[sample_index]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]
    time_axis = pd.to_datetime(df.loc[mask, 'date_time'].values[:96][is_daytime])

    sample_true = test_Y[sample_index][is_daytime]
    sample_pred = preds[sample_index][is_daytime]
    time_str = pd.to_datetime(ts).strftime("%Y-%m-%d")

    # 动态时间轴格式化
    ax = plt.gca()
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    ax.xaxis.set_major_locator(mdates.HourLocator(interval=3))

    # 增强可视化元素
    plt.plot(time_axis, sample_true, label='实际功率',
             color='#2c7bb6', lw=2, marker='o', markersize=6, zorder=3)
    plt.plot(time_axis, sample_pred, label='预测功率',
             color='#d7191c', lw=2, linestyle='--', zorder=2)

    # 误差带填充

```

```

plt.fill_between(time_axis, sample_pred, sample_true,
                 where=sample_pred > sample_true,
                 facecolor='#fdae61', alpha=0.3, label='正向误差')
plt.fill_between(time_axis, sample_pred, sample_true,
                 where=sample_pred <= sample_true,
                 facecolor='#abdda4', alpha=0.3, label='负向误差')

# 统计标注
rmse = np.sqrt(mean_squared_error(sample_true, sample_pred))
mae = mean_absolute_error(sample_true, sample_pred)
plt.text(0.02, 0.95, f'RMSE: {rmse:.2f} MW\nMAE: {mae:.2f} MW',
        transform=ax.transAxes, fontsize=12,
        bbox=dict(facecolor='white', alpha=0.8))

# 图例与装饰
plt.title(f'专业预测可视化 - {time_str}', pad=20)
plt.xlabel('时间 (15 分钟间隔)')
plt.ylabel('功率 (MW)')
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=3)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()

def plot_error_analysis(preds, test_Y):
    """ 多维误差分析矩阵 """
    errors = preds - test_Y
    errors_flat = errors.flatten()

    fig = plt.figure(figsize=(16, 12), constrained_layout=True)
    gs = fig.add_gridspec(3, 2)

    # 误差分布
    ax1 = fig.add_subplot(gs[0, 0])
    sns.histplot(x=errors_flat, kde=True, ax=ax1, bins=30,
                color='#2c7bb6', edgecolor='white')
    ax1.axvline(np.mean(errors_flat), color='red', linestyle='--')
    ax1.set_title("误差分布分析", fontsize=14, pad=15)

    # 误差热力图
    ax2 = fig.add_subplot(gs[0, 1])
    sns.heatmap(errors[:28], cmap="coolwarm", center=0,
                ax=ax2, cbar_kws={'label': '误差 (MW)'})
    ax2.set_title("误差时间分布模式 (前 28 个样本)", fontsize=14)

```



```

# Q-Q 图
ax3 = fig.add_subplot(gs[1, 0])
stats.probplot(errors_flat, dist="norm", plot=ax3)
ax3.get_lines()[0].set_markerfacecolor('#2c7bb6')
ax3.title.set_text('正态性检验 Q-Q 图')

# 残差分析
ax4 = fig.add_subplot(gs[1, 1])
sns.residplot(x=test_Y.flatten(), y=preds.flatten(),
              lowess=True, color='#2c7bb6',
              line_kws={'color': 'red'}, ax=ax4)
ax4.set_title("残差分析图", fontsize=14)

# 综合统计
plt.suptitle("多维误差分析报告", y=1.02, fontsize=16)
plt.show()

def interactive_forecast_plot(preds, test_Y, test_timestamps, df, sample_index=0):
    """交互式可视化（需安装plotly）"""
    ts = test_timestamps[sample_index]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))
    time_points = df.loc[mask, 'date_time'].values
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]

    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=time_points[is_daytime],
        y=test_Y[sample_index][is_daytime],
        name='实际功率',
        line=dict(color='#1f77b4', width=3))
    )
    fig.add_trace(go.Scatter(
        x=time_points[is_daytime],
        y=preds[sample_index][is_daytime],
        name='预测功率',
        line=dict(color='#ff7f0e', width=3, dash='dot'))
    )

    fig.update_layout(
        title=f'交互式预测可视化 - {ts.strftime("%Y-%m-%d")}',
        xaxis_title='时间',
        yaxis_title='功率 (MW)',

```

```

        hovermode="x unified",
        template='plotly_white',
        height=500,
        margin=dict(l=50, r=50, b=80, t=80),
        annotations=[
            dict(
                text=f"样本编号: {sample_index}",
                x=0.05, y=0.95,
                showarrow=False,
                xref="paper", yref="paper"
            )
        ]
    )
fig.show()

if __name__ == "__main__":
    model_dict = {
        "BiFusionModel": BiFusionModel(input_len=96), # 你需要预先定义这个
        模型结构
        "PureLSTM": PureLSTM(input_len=96),
        "FusionModel": FusionModel(input_len=96),
        # "BiFusionModel": BiFusionModel(input_len=96) # 你需要预先定义
        这个模型结构
    }

    predictions = {}
    metrics_all = {}

    for name, model in model_dict.items():
        print(f"\n===== 正在训练模型: {name} =====")
        use_mv = True # True: 问题 3, False: 问题 2
        X_train, X_test = get_inputs(use_mv)

        model, max_power = train_model(model, X_train, train_Y)
        preds, rmse, mae, mape = evaluate_model(model, X_test, test_Y,
        test_timestamps, df, max_power)

        metrics = compute_all_metrics(test_Y, preds, test_timestamps, df)
        predictions[name] = preds
        metrics_all[name] = metrics

    # 每个模型分别导出预测表格
    export_prediction_table(preds, test_Y, test_timestamps, df,

```

```

method_name=f"{name}预测功率", output_file=f"3prediction_{name}.csv")

    # 每个模型分别可视化
    print(f"\n>> 可视化: {name}")
    visualize_predictions(preds, test_Y, test_timestamps, df)
    plot_professional_forecast(preds, test_Y, test_timestamps, df)
    plot_error_analysis(preds, test_Y)

# ===== 指标对比表格 =====
df_metrics = pd.DataFrame(metrics_all).T
print("\n3 三模型评估指标对比: ")
print(df_metrics.round(4))

# 可视化热图
plt.figure(figsize=(10, 6))
sns.heatmap(df_metrics, annot=True, fmt=".3f", cmap="YlGnBu")
plt.title("三模型评估指标热力图 (白昼时段)")
plt.tight_layout()
plt.show()

# ===== 三模型每日对比图 (默认 index = 0) =====
day_idx = 0
ts = test_timestamps[day_idx]
plt.figure(figsize=(12, 6))
for name in predictions:
    pred = predictions[name][day_idx]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]
    time_axis = df.loc[mask, 'date_time'].values[:96][is_daytime]
    plt.plot(time_axis, pred[is_daytime], label=f"{name} 预测")

# 添加真实值
true = test_Y[day_idx]
plt.plot(time_axis, true[is_daytime], label=" 真实功率 ", linestyle='--',
linewidth=2, color='black')
# plt.title(f"每日预测对比图 (白昼) | 起报时间: {ts.date()}")
plt.title(f"每日预测对比图 (白昼) | 起报时间: {pd.to_datetime(ts).date()}")

plt.xlabel("时间")
plt.ylabel("功率 (MW)")
plt.legend()
plt.grid(True)
plt.tight_layout()

```

```

plt.show()

# ===== 统一输出预测对比表格 =====
records = []
for i, ts in enumerate(test_timestamps):
    start_time = pd.to_datetime(ts)
    for j in range(96):
        forecast_time = start_time + pd.Timedelta(days=1) +
pd.Timedelta(minutes=15 * j)
        real_val_row = df[df['date_time'] == forecast_time]
        actual_power = real_val_row['power'].values[0] if not
real_val_row.empty else np.nan

        row = {
            "起报时间": start_time.replace(hour=0, minute=0, second=0),
            "预报时间": forecast_time,
            "实际功率": actual_power
        }
        for name in predictions:
            row[f"{name} 预测功率"] = predictions[name][i, j]
        records.append(row)

df_all_preds = pd.DataFrame(records)
df_all_preds.to_csv("3 三模型预测结果对比表.csv", index=False,
encoding='utf-8-sig')
print("\n 已保存统一预测对比表格为：3 三模型预测结果对比表.csv")

# 结束计时
end_time = time.time()
elapsed_time = end_time - start_time

print(f"代码执行时间: {elapsed_time:.4f} 秒")

问题 4
# -*- coding: utf-8 -*-
"""
Created on 2025/5/25 09:13

@author: Prince
"""
import time

# 开始计时
start_time = time.time()

```

```

import numpy as np
import pandas as pd
import matplotlib.pyplot as plt
from scipy.integrate import odeint
from sklearn.preprocessing import MinMaxScaler
from matplotlib import rcParams

# ===== 新增库导入 =====
import matplotlib.dates as mdates
from scipy import stats
import plotly.express as px
import plotly.graph_objects as go
import seaborn as sns
# =====
#
# rcParams['font.family'] = 'SimHei'
# rcParams['axes.unicode_minus'] = False

import matplotlib.pyplot as plt
plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文支持字体
plt.rcParams['axes.unicode_minus'] = False # 修复负号显示问题

# ===== 全局样式设置 =====
# plt.style.use('seaborn-whitegrid') # 注释此行，删除无效样式调用
sns.set_theme(style='whitegrid') # 保留 Seaborn 的白色网格样式配置
rcParams.update({
    'figure.figsize': (10, 5),
    'axes.titlesize': 14,
    'axes.labelsize': 12,
    'xtick.labelsize': 10,
    'ytick.labelsize': 10,
    'grid.linestyle': '--',
    'figure.dpi': 150
})
# =====
df = pd.read_csv("station00.csv")

df['date_time'] = pd.to_datetime(df['date_time'])

# 添加日期信息
df['month'] = df['date_time'].dt.month
df['day'] = df['date_time'].dt.day
df['date'] = df['date_time'].dt.date

```

```

df['date_time'] = pd.to_datetime(df['date_time'])
df['date'] = df['date_time'].dt.date

# ===== 提前放置：先生成 test_dates =====
unique_dates = df['date'].drop_duplicates().reset_index(drop=True)
test_months = [2, 5, 8, 11]
test_dates = []
for m in test_months:
    month_dates = unique_dates[unique_dates.map(lambda d: d.month) == m]
    if len(month_dates) >= 7:
        test_dates.extend(month_dates[-7:].tolist())

# ===== 然后再使用 test_dates =====
df['set'] = df['date'].apply(lambda d: 'test' if d in test_dates else 'train')
df['is_daytime'] = df['power'] > 0.05

# 添加风向分解特征
df['wind_x'] = np.cos(np.radians(df['nwp_winddirection']))
df['wind_y'] = np.sin(np.radians(df['nwp_winddirection']))
df.drop(columns=['nwp_winddirection'], inplace=True)

# 选定用于模型的气象变量
features = ['power', 'nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
            'nwp_humidity', 'nwp_windspeed', 'nwp_pressure', 'wind_x', 'wind_y']

# 归一化（仅对训练集拟合，全部样本变换）
from sklearn.preprocessing import MinMaxScaler

scalers = {}
for col in features:
    scaler = MinMaxScaler()
    train_vals = df[df['set'] == 'train'][col].values.reshape(-1, 1)
    df[col + '_scaled'] = scaler.fit_transform(df[[col]])
    scalers[col] = scaler

scaled_features = [col + '_scaled' for col in features]

# 指定测试集：2、5、8、11 月最后 7 天
unique_dates = df['date'].drop_duplicates().reset_index(drop=True)
test_months = [2, 5, 8, 11]
test_dates = []
for m in test_months:

```

```

month_dates = unique_dates[unique_dates.map(lambda d: d.month) == m]
if len(month_dates) >= 7:
    test_dates.extend(month_dates[-7:].tolist())

# ===== 新增模块：多变量样本构造函数 =====
def construct_multivariate_samples(df, scaled_features, input_len=96,
forecast_len=96):
    train_X, train_Y, test_X, test_Y, test_timestamps = [], [], [], [], []
    values = df[scaled_features].values
    powers = df['power'].values
    flags = df['set'].values
    timestamps = df['date_time'].values

    for i in range(len(df) - input_len - forecast_len):
        X_seq = values[i:i + input_len]
        Y_seq = powers[i + input_len: i + input_len + forecast_len]
        ts_flag = flags[i + input_len + forecast_len - 1]
        ts_point = timestamps[i + input_len]

        if flags[i] == ts_flag == 'train':
            train_X.append(X_seq)
            train_Y.append(Y_seq)
        elif flags[i] == ts_flag == 'test':
            test_X.append(X_seq)
            test_Y.append(Y_seq)
            test_timestamps.append(ts_point)

    return (np.array(train_X), np.array(train_Y),
            np.array(test_X), np.array(test_Y),
            np.array(test_timestamps))

# 替换原始一维样本构造逻辑
scaled_features = [col + '_scaled' for col in features] # 9 个特征列名
train_X_mv, train_Y, test_X_mv, test_Y, test_timestamps =
construct_multivariate_samples(
    df, scaled_features, input_len=96, forecast_len=96)

# 将多变量返回值直接覆盖原变量名（兼容后续代码）
train_X = train_X_mv # shape=(N, 96, 9)
test_X = test_X_mv # shape=(M, 96, 9)
# train_Y/test_Y/test_timestamps 变量名已兼容

```

```

# 可在训练流程中控制是否启用 NWP 数据
def get_inputs(use_mv):
    if use_mv:
        return train_X_mv, test_X_mv  # 多变量输入 shape=(N, 96, 9)
    else:
        return train_X[..., None], test_X[..., None]  # 单变量输入 shape=(N, 96,
1)

import torch
import torch.nn as nn

class LSTMBranch(nn.Module):
    def __init__(self, input_len, hidden_dim=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(1, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, input_len)

    def forward(self, x):
        x = x.unsqueeze(-1)
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

class TCNBlock(nn.Module):
    def __init__(self, in_channels, out_channels, kernel_size=3, dilation=1):
        super().__init__()
        self.conv = nn.Conv1d(in_channels, out_channels, kernel_size,
                               padding=(kernel_size - 1) * dilation // 2,
                               dilation=dilation)
        self.bn = nn.BatchNorm1d(out_channels)
        self.relu = nn.ReLU()

    def forward(self, x):
        return self.relu(self.bn(self.conv(x)))

class TCNBranch(nn.Module):
    def __init__(self, input_len):
        super().__init__()
        self.net = nn.Sequential(
            TCNBlock(1, 16, dilation=1),
            TCNBlock(16, 32, dilation=2),
            TCNBlock(32, 64, dilation=4)
        )
        self.fc = nn.Linear(64, input_len)

```



```

def forward(self, x):
    x = x.unsqueeze(1)
    x = self.net(x)
    return self.fc(x.mean(dim=2))

class MLPBranch(nn.Module):
    def __init__(self, input_len):
        super().__init__()
        self.fc = nn.Sequential(
            nn.Linear(input_len, 64),
            nn.ReLU(),
            nn.Linear(64, 64),
            nn.ReLU(),
            nn.Linear(64, input_len)
        )

    def forward(self, x):
        return self.fc(x)

# 修改 FusionModel 中 3 个子模块：使用 x[:, :, 0] 提取功率输入进行兼容
class FusionModel(nn.Module):
    def __init__(self, input_len=96):
        super().__init__()
        self.lstm_branch = LSTMBranch(input_len)
        self.tcn_branch = TCNBranch(input_len)
        self.mlp_branch = MLPBranch(input_len)
        self.attn = nn.Sequential(
            nn.Linear(3 * input_len, 128), nn.ReLU(), nn.Linear(128, 3),
nn.Softmax(dim=1))
        self.output_layer = nn.Linear(input_len, input_len)

    def forward(self, x):
        x1d = x[:, :, 0] # 提取功率作为单通道用于三支路兼容
        lstm_out = self.lstm_branch(x1d)
        tcn_out = self.tcn_branch(x1d)
        mlp_out = self.mlp_branch(x1d)
        concat = torch.cat([lstm_out, tcn_out, mlp_out], dim=1)
        weights = self.attn(concat)
        out = (weights[:, 0:1] * lstm_out + weights[:, 1:2] * tcn_out + weights[:, 2:3]
* mlp_out)
        return self.output_layer(out)

# ===== 模型定义区：新增两种模型

```

```

=====

# ===== 模型输入调整 =====
# 所有模型前向传播 now uses x.shape = (batch, 96, num_feat)

# 修改 PureLSTM
class PureLSTM(nn.Module):
    def __init__(self, input_len=96, input_dim=9, hidden_dim=64, num_layers=2):
        super().__init__()
        self.lstm = nn.LSTM(input_dim, hidden_dim, num_layers, batch_first=True)
        self.fc = nn.Linear(hidden_dim, input_len)

    def forward(self, x):
        out, _ = self.lstm(x)
        return self.fc(out[:, -1, :])

# 2. BiLSTM + TCN + Transformer (BiFusion)
class TransformerBlock(nn.Module):
    def __init__(self, input_dim, nhead=4, num_layers=2):
        super().__init__()
        encoder_layer = nn.TransformerEncoderLayer(d_model=input_dim,
nhead=nhead, batch_first=True)
        self.transformer = nn.TransformerEncoder(encoder_layer,
num_layers=num_layers)

    def forward(self, x):
        return self.transformer(x)

# 修改 BiFusionModel
class BiFusionModel(nn.Module):
    def __init__(self, input_len=96, input_dim=9):
        super().__init__()
        self.bilstm = nn.LSTM(input_dim, 64, batch_first=True, bidirectional=True)
        self.tcn = nn.Sequential(
            TCNBlock(1, 32, dilation=1),
            TCNBlock(32, 64, dilation=2)
        )
        self.input_proj = nn.Linear(input_dim, 64) # <<< 添加此层
        self.transformer = TransformerBlock(input_dim=64)
        self.fc = nn.Sequential(
            nn.Linear(64 * 2 + 64 + 64, 128), nn.ReLU(), nn.Linear(128,
input_len))

```

```

def forward(self, x):
    bilstm_feat = self.bilstm(x)[0][:, -1, :]
    tcn_feat = self.tcn(x[:, :, 0].unsqueeze(1)).mean(dim=2)
    trans_feat = self.transformer(self.input_proj(x)).mean(dim=1)  # <<< 添
加映射
    feat = torch.cat([bilstm_feat, tcn_feat, trans_feat], dim=1)
    return self.fc(feat)

import torch
import torch.nn as nn
import numpy as np
from torch.utils.data import DataLoader, TensorDataset

def train_model(model, train_X, train_Y, val_split=0.1, batch_size=64, epochs=50,
lr=0.001, patience=5):
    # 设置设备
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.to(device)

    # 数据归一化
    max_power = train_X.max()
    train_X, train_Y = train_X / max_power, train_Y / max_power

    # 创建验证集
    val_size = int(len(train_X) * val_split)
    idx = np.random.permutation(len(train_X))
    val_idx, train_idx = idx[:val_size], idx[val_size:]

    # 转换为 PyTorch 张量
    X_train = torch.tensor(train_X[train_idx], dtype=torch.float32)
    Y_train = torch.tensor(train_Y[train_idx], dtype=torch.float32)
    X_val = torch.tensor(train_X[val_idx], dtype=torch.float32)
    Y_val = torch.tensor(train_Y[val_idx], dtype=torch.float32)

    # 创建数据加载器
    train_loader = DataLoader(TensorDataset(X_train, Y_train),
batch_size=batch_size, shuffle=True)
    val_loader = DataLoader(TensorDataset(X_val, Y_val), batch_size=batch_size)

    # 定义优化器和损失函数
    optimizer = torch.optim.Adam(model.parameters(), lr=lr)

```

```

criterion = nn.MSELoss()
best_loss, counter = float('inf'), 0

# 创建保存模型的目录
import os
os.makedirs('models', exist_ok=True)
model_path = 'models/best_model.pth'

# 训练循环
for epoch in range(epochs):
    # 训练阶段
    model.train()
    total_loss = 0
    for xb, yb in train_loader:
        xb, yb = xb.to(device), yb.to(device)
        optimizer.zero_grad()
        loss = criterion(model(xb), yb)
        loss.backward()
        optimizer.step()
        total_loss += loss.item() * xb.size(0)

    # 验证阶段
    val_loss = 0
    model.eval()
    with torch.no_grad():
        for xb, yb in val_loader:
            xb, yb = xb.to(device), yb.to(device)
            val_loss += criterion(model(xb), yb).item() * xb.size(0)

    # 计算平均损失
    train_loss = total_loss / len(train_loader.dataset)
    val_loss /= len(val_loader.dataset)
    print(f'Epoch {epoch + 1}: Train={train_loss:.4f}, Val={val_loss:.4f}')

    # 早停机制
    if val_loss < best_loss:
        best_loss = val_loss
        counter = 0
        torch.save(model.state_dict(), model_path)
        print(f'模型保存于 {model_path}')
    else:
        counter += 1
        if counter >= patience:
            print(f'早停触发, 最佳验证损失: {best_loss:.4f}')

```

```

        break

    # 加载最佳模型
    model.load_state_dict(torch.load(model_path))
    return model, max_power

# ----- 预测与评估 -----
from sklearn.metrics import mean_squared_error, mean_absolute_error

def evaluate_model(model, test_X, test_Y, test_timestamps, df, max_power):
    device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
    model.eval()

    # 标准化输入
    X_tensor = torch.tensor(test_X / max_power, dtype=torch.float32).to(device)

    # 执行预测
    with torch.no_grad():
        preds = model(X_tensor).cpu().numpy() * max_power # 反归一化
    true = test_Y

    # 评估白昼指标（只在白昼时间段评估误差）
    day_indices = []
    for ts in test_timestamps:
        day_mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
        is_daytime = df.loc[day_mask, 'is_daytime'].values
        day_indices.append(is_daytime)

    all_preds, all_true = [], []
    for pred, real, mask in zip(preds, true, day_indices):
        mask = mask[:96] # 确保掩码长度一致
        all_preds.extend(pred[mask])
        all_true.extend(real[mask])

    rmse = np.sqrt(mean_squared_error(all_true, all_preds))
    mae = mean_absolute_error(all_true, all_preds)
    mape = np.mean(np.abs((np.array(all_true) - np.array(all_preds)) /
(np.array(all_true) + 1e-5))) * 100

    print(f'白昼 RMSE: {rmse:.4f} MAE: {mae:.4f} MAPE: {mape:.2f}%')

    return preds, rmse, mae, mape

```

```

def compute_all_metrics(true_values, pred_values, timestamps, df,
capacity_kW=6600):
    """
    计算附件 1 中的所有误差评估指标
    :param true_values: np.array, shape = (N, 96)
    :param pred_values: np.array, shape = (N, 96)
    :param timestamps: list of datetime 起报时间
    :param df: 原始 dataframe (用于白昼判断)
    :param capacity_kW: 单位为千瓦 (默认 6600)
    """
    C = capacity_kW / 1000.0 # 转为 MW
    all_true, all_pred = [], []

    for i, ts in enumerate(timestamps):
        mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
        is_day = df.loc[mask, 'is_daytime'].values[:96]
        pred = pred_values[i][is_day]
        true = true_values[i][is_day]

        all_true.extend(true)
        all_pred.extend(pred)

    all_true = np.array(all_true)
    all_pred = np.array(all_pred)

    # 附件 1 公式指标计算
    err = (all_true - all_pred) / C
    abs_err = np.abs(err)

    E_rmse = np.sqrt(np.mean(err**2))
    E_mae = np.mean(abs_err)
    E_me = np.mean(err)
    r = np.corrcoef(all_true, all_pred)[0, 1]
    C_R = (1 - E_rmse) * 100
    Q_R = np.mean(abs_err < 0.25) * 100

    print(f'【附件 1 考核指标】基于装机容量 {C:.2f} MW")
    print(f'E_rmse: {E_rmse:.4f}")
    print(f'E_mae : {E_mae:.4f}")
    print(f'E_me : {E_me:.4f}")
    print(f'r : {r:.4f}")
    print(f'C_R : {C_R:.2f}%")

```

```

print(f'Q_R      : {Q_R:.2f}%")

return {
    'E_rmse': E_rmse,
    'E_mae': E_mae,
    'E_me': E_me,
    'r': r,
    'C_R': C_R,
    'Q_R': Q_R
}

# ----- 使用示例 -----
# 训练模型
# model = FusionModel(input_len=96)
# model, max_power = train_model(model, train_X, train_Y)
#
## 评估测试集
# preds, rmse, mae, mape = evaluate_model(model, test_X, test_Y, test_timestamps,
# df, max_power)

import pandas as pd

def export_prediction_table(preds, test_Y, test_timestamps, df, method_name='融合
模型预测', output_file='prediction_table.csv'):
    """
    将模型预测结果展开为标准提交表格格式
    :param preds: 模型预测值 (N, 96)
    :param test_Y: 实际值 (N, 96)
    :param test_timestamps: 起报时间列表 (应为目标日 00:00:00)
    :param df: 原始 DataFrame, 含真实功率
    :param method_name: 列标题中方法名
    :param output_file: 保存文件路径
    """
    records = []
    df = df.copy()
    df['date_time'] = pd.to_datetime(df['date_time'])

    for i, ts in enumerate(test_timestamps):
        start_time = pd.to_datetime(ts)
        for j in range(96):
            forecast_time = start_time + pd.Timedelta(days=1) +
pd.Timedelta(minutes=15 * j)

            # 查找实际功率
            real_val_row = df[df['date_time'] == forecast_time]

```

```

        if not real_val_row.empty:
            actual_power = real_val_row['power'].values[0]
        else:
            actual_power = np.nan # 若没有记录, 用 nan 占位

        records.append({
            "起报时间": start_time.replace(hour=0, minute=0, second=0),
            "预报时间": forecast_time,
            "实际功率 (MW)": actual_power,
            f"{method_name} (MW)": preds[i, j]
        })

df_pred = pd.DataFrame(records)
df_pred.to_csv(output_file, index=False, encoding='utf-8-sig')
print(f'预测结果已保存至 {output_file}')

# 使用方法
# 计算所有指标
# export_prediction_table(preds, test_Y, test_timestamps, df, method_name='问题 2
融合模型预测')

# metrics = compute_all_metrics(test_Y, preds, test_timestamps, df,
capacity_kW=6600)

import matplotlib.pyplot as plt
import seaborn as sns

def visualize_predictions(preds, test_Y, test_timestamps, df):
    # 设置全局绘图风格
    # plt.style.use('seaborn-whitegrid') # 删除这一行或注释掉
    sns.set_theme(style='whitegrid') # seaborn 本身即可设定风格

    sns.set_context("notebook", font_scale=1.1)
    # 原代码开头添加字体配置
    import matplotlib.pyplot as plt
    plt.rcParams['font.sans-serif'] = ['SimHei'] # 设置中文字体
    plt.rcParams['axes.unicode_minus'] = False # 解决负号问题

    # ----- 1. 可视化第一个样本的白昼时段功率曲线 -----
    sample_index = 0
    ts = test_timestamps[sample_index]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))

```



```

is_daytime = df.loc[mask, 'is_daytime'].values[:96]
time_axis = df.loc[mask, 'date_time'].values[:96][is_daytime]

sample_pred = preds[sample_index][is_daytime]
sample_true = test_Y[sample_index][is_daytime]

plt.figure(figsize=(12, 5))
plt.plot(time_axis, sample_true, label='真实功率', linewidth=2, color='royalblue')
plt.plot(time_axis, sample_pred, label='预测功率', linewidth=2, linestyle='--',
color='darkorange')
plt.fill_between(time_axis, sample_true, sample_pred, color='gray', alpha=0.3,
label='误差区域')
plt.title(f'白昼时段预测曲线（起报时间：{ts}）')
plt.xlabel('时间')
plt.ylabel('功率 (MW)')
plt.legend()
plt.tight_layout()
plt.grid(True)
plt.show()

# ----- 2. 所有白昼误差分布图 -----
all_true, all_pred = [], []
for i, ts in enumerate(test_timestamps):
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts +
pd.Timedelta(days=1))
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]
    all_true.extend(test_Y[i][is_daytime])
    all_pred.extend(preds[i][is_daytime])
errors = np.array(all_pred) - np.array(all_true)

plt.figure(figsize=(10, 4))
sns.histplot(errors, kde=True, bins=50, color='teal')
plt.axvline(x=0, color='red', linestyle='--')
plt.title('预测误差分布（白昼时段）')
plt.xlabel('误差（预测 - 实际）(MW)')
plt.ylabel('频数')
plt.tight_layout()
plt.show()

# ----- 3. 散点图：预测值 vs 实际值 -----
plt.figure(figsize=(6, 6))
sns.scatterplot(x=all_true, y=all_pred, alpha=0.5, edgecolor=None)
max_val = max(max(all_true), max(all_pred))
plt.plot([0, max_val], [0, max_val], linestyle='--', color='red', label='理想预测线')

```

```

plt.xlabel('实际功率 (MW)')
plt.ylabel('预测功率 (MW)')
plt.title('预测 vs 实际 (白昼)')
plt.legend()
plt.tight_layout()
plt.grid(True)
plt.show()

def plot_professional_forecast(preds, test_Y, test_timestamps, df, sample_index=0):
    """ 专业级预测曲线可视化 """
    plt.figure(figsize=(14, 6))

    # 数据准备
    ts = test_timestamps[sample_index]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]
    time_axis = pd.to_datetime(df.loc[mask, 'date_time'].values[:96][is_daytime])

    sample_true = test_Y[sample_index][is_daytime]
    sample_pred = preds[sample_index][is_daytime]
    time_str = pd.to_datetime(ts).strftime("%Y-%m-%d")

    # 动态时间轴格式化
    ax = plt.gca()
    ax.xaxis.set_major_formatter(mdates.DateFormatter("%H:%M"))
    ax.xaxis.set_major_locator(mdates.HourLocator(interval=3))

    # 增强可视化元素
    plt.plot(time_axis, sample_true, label='实际功率',
             color='#2c7bb6', lw=2, marker='o', markersize=6, zorder=3)
    plt.plot(time_axis, sample_pred, label='预测功率',
             color='#d7191c', lw=2, linestyle='--', zorder=2)

    # 误差带填充
    plt.fill_between(time_axis, sample_pred, sample_true,
                     where=sample_pred > sample_true,
                     facecolor='#fdae61', alpha=0.3, label='正向误差')
    plt.fill_between(time_axis, sample_pred, sample_true,
                     where=sample_pred <= sample_true,
                     facecolor='#abdda4', alpha=0.3, label='负向误差')

    # 统计标注
    rmse = np.sqrt(mean_squared_error(sample_true, sample_pred))

```

```

mae = mean_absolute_error(sample_true, sample_pred)
plt.text(0.02, 0.95, f'RMSE: {rmse:.2f} MW\nMAE: {mae:.2f} MW',
         transform=ax.transAxes, fontsize=12,
         bbox=dict(facecolor='white', alpha=0.8))

# 图例与装饰
plt.title(f'专业预测可视化 - {time_str}', pad=20)
plt.xlabel('时间 (15 分钟间隔)')
plt.ylabel('功率 (MW)')
plt.legend(loc='upper center', bbox_to_anchor=(0.5, -0.15), ncol=3)
plt.grid(True, alpha=0.4)
plt.tight_layout()
plt.show()

def plot_error_analysis(preds, test_Y):
    """ 多维误差分析矩阵 """
    errors = preds - test_Y
    errors_flat = errors.flatten()

    fig = plt.figure(figsize=(16, 12), constrained_layout=True)
    gs = fig.add_gridspec(3, 2)

    # 误差分布
    ax1 = fig.add_subplot(gs[0, 0])
    sns.histplot(x=errors_flat, kde=True, ax=ax1, bins=30,
                 color='#2c7bb6', edgecolor='white')
    ax1.axvline(np.mean(errors_flat), color='red', linestyle='--')
    ax1.set_title("误差分布分析", fontsize=14, pad=15)

    # 误差热力图
    ax2 = fig.add_subplot(gs[0, 1])
    sns.heatmap(errors[:28], cmap="coolwarm", center=0,
                 ax=ax2, cbar_kws={'label': '误差 (MW)'})
    ax2.set_title("误差时间分布模式 (前 28 个样本)", fontsize=14)

    # Q-Q 图
    ax3 = fig.add_subplot(gs[1, 0])
    stats.probplot(errors_flat, dist="norm", plot=ax3)
    ax3.get_lines()[0].set_markerfacecolor('#2c7bb6')
    ax3.title.set_text('正态性检验 Q-Q 图')

    # 残差分析
    ax4 = fig.add_subplot(gs[1, 1])

```

```

sns.residplot(x=test_Y.flatten(), y=preds.flatten(),
              lowess=True, color='#2c7bb6',
              line_kws={'color': 'red'}, ax=ax4)
ax4.set_title("残差分析图", fontsize=14)

# 综合统计
plt.suptitle("多维误差分析报告", y=1.02, fontsize=16)
plt.show()

def interactive_forecast_plot(preds, test_Y, test_timestamps, df, sample_index=0):
    """交互式可视化（需安装plotly）"""
    ts = test_timestamps[sample_index]
    mask = (df['date_time'] >= ts) & (df['date_time'] < ts + pd.Timedelta(days=1))
    time_points = df.loc[mask, 'date_time'].values
    is_daytime = df.loc[mask, 'is_daytime'].values[:96]

    fig = go.Figure()
    fig.add_trace(go.Scatter(
        x=time_points[is_daytime],
        y=test_Y[sample_index][is_daytime],
        name='实际功率',
        line=dict(color='#1f77b4', width=3))
    )
    fig.add_trace(go.Scatter(
        x=time_points[is_daytime],
        y=preds[sample_index][is_daytime],
        name='预测功率',
        line=dict(color='#ff7f0e', width=3, dash='dot'))
    )

    fig.update_layout(
        title=f'交互式预测可视化 - {ts.strftime("%Y-%m-%d")}',
        xaxis_title='时间',
        yaxis_title='功率 (MW)',
        hovermode="x unified",
        template='plotly_white',
        height=500,
        margin=dict(l=50, r=50, b=80, t=80),
        annotations=[
            dict(
                text=f'样本编号: {sample_index}',
                x=0.05, y=0.95,
                showarrow=False,

```

```

        xref="paper", yref="paper"
    )
]
)
fig.show()

def run_input_mode_experiments(df, model_dict, modes=['nwp', 'lmd', 'mixed']):
    from collections import defaultdict

    results = defaultdict(dict)

    def get_feature_columns(mode):
        if mode == 'nwp':
            return ['nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
                    'nwp_humidity', 'nwp_windspeed', 'nwp_pressure', 'wind_x',
                    'wind_y']
        elif mode == 'lmd':
            return ['lmd_totalirrad', 'lmd_diffuseirrad', 'lmd_temperature',
                    'lmd_pressure', 'lmd_windspeed', 'lmd_winddirection']
        elif mode == 'mixed':
            return ['nwp_globalirrad', 'nwp_directirrad', 'nwp_temperature',
                    'nwp_humidity', 'nwp_windspeed', 'nwp_pressure', 'wind_x',
                    'wind_y',
                    'lmd_totalirrad', 'lmd_diffuseirrad', 'lmd_temperature',
                    'lmd_pressure', 'lmd_windspeed', 'lmd_winddirection']
        else:
            raise ValueError("未知输入模式")

    for mode in modes:
        print(f"\n===== 正在处理输入模式: {mode.upper()} =====")
        features = get_feature_columns(mode)
        scaled_features = []

        # 归一化当前字段
        for col in features + ['power']:
            scaler = MinMaxScaler()
            train_vals = df[df['set'] == 'train'][col].values.reshape(-1, 1)
            df[col + '_scaled'] = scaler.fit_transform(df[[col]])
            scaled_features.append(col + '_scaled')

        # 构造样本
        train_X_mv, train_Y, test_X_mv, test_Y, test_timestamps =
construct_multivariate_samples(

```

```

df, scaled_features, input_len=96, forecast_len=96)

for name, model in model_dict.items():
    print(f"\n>> 模型: {name} | 输入: {mode}")
    input_dim = len(scaled_features)
    if name == "BiFusionModel":
        model_instance = BiFusionModel(input_len=96,
input_dim=input_dim)
    elif name == "PureLSTM":
        model_instance = PureLSTM(input_len=96,
input_dim=input_dim)
    elif name == "FusionModel":
        model_instance = FusionModel(input_len=96)

    model_trained, max_power = train_model(model_instance,
train_X_mv, train_Y)
    preds, rmse, mae, mape = evaluate_model(model_trained, test_X_mv,
test_Y, test_timestamps, df, max_power)

    metrics = compute_all_metrics(test_Y, preds, test_timestamps, df)

    results[f'{name}_{mode}'] = {
        'RMSE': rmse, 'MAE': mae, 'MAPE': mape,
        **metrics
    }

# 输出结果表格和图像
export_prediction_table(preds, test_Y, test_timestamps, df,
                        method_name=f'{name}_{mode}',
output_file=f'Q4_pred_{name}_{mode}.csv")

# 可选: 开启以下三行可视化
visualize_predictions(preds, test_Y, test_timestamps, df)
plot_professional_forecast(preds, test_Y, test_timestamps, df)
plot_error_analysis(preds, test_Y)

# 输出热力图对比
df_results = pd.DataFrame(results).T
print("\n===== 空间降尺度输入对比结果表 =====")
print(df_results.round(4))

plt.figure(figsize=(12, 6))
sns.heatmap(df_results, annot=True, fmt=".3f", cmap="YlGnBu")
plt.title("问题 4: 三种输入配置+多模型精度对比 (白昼指标)")

```

```

plt.tight_layout()
plt.show()

df_results.to_csv("Q4_模型输入对比结果.csv", encoding='utf-8-sig')
print("结果已保存： Q4_模型输入对比结果.csv")

return df_results

import matplotlib.pyplot as plt
import seaborn as sns
import numpy as np

def visualize_q4_results(df_results):
    import matplotlib.cm as cm

    plt.rcParams['font.sans-serif'] = ['SimHei']
    plt.rcParams['axes.unicode_minus'] = False

    models = sorted({idx.split('_')[0] for idx in df_results.index})
    modes = sorted({idx.split('_')[1] for idx in df_results.index})
    metrics = ['RMSE', 'MAE', 'MAPE', 'C_R', 'Q_R', 'E_rmse']

    cmap = cm.get_cmap("tab10", len(models))

    # ===== 图 1： 热力图 =====
    plt.figure(figsize=(12, 6))
    sns.heatmap(df_results[metrics], annot=True, fmt=".3f", cmap="YlOrBr")
    plt.title("三输入 + 多模型空间降尺度预测指标热力图")
    plt.tight_layout()
    plt.show()

    # ===== 图 2： 雷达图（每个模型下不同输入） =====
    from math import pi
    for model in models:
        plt.figure(figsize=(6, 6))
        angles = [n / float(len(metrics)) * 2 * pi for n in range(len(metrics))]
        angles += angles[:1] # 闭合

        ax = plt.subplot(111, polar=True)
        for i, mode in enumerate(modes):
            key = f'{model}_{mode}'
            values = df_results.loc[key, metrics].tolist()
            values += values[:1]
            ax.plot(angles, values, label=mode.upper(), linewidth=2)

```

```

ax.fill(angles, values, alpha=0.15)

ax.set_xticks(angles[:-1])
ax.set_xticklabels(metrics)
ax.set_title(f'{model}模型 - 输入对比雷达图', y=1.1)
plt.legend(loc='upper right', bbox_to_anchor=(1.3, 1.1))
plt.tight_layout()
plt.show()

# ===== 图 3: 误差柱状图（各输入配置下模型对比） =====
for metric in ['RMSE', 'MAPE', 'E_rmse']:
    plt.figure(figsize=(10, 5))
    for i, mode in enumerate(modes):
        data = [df_results.loc[f'{model}_{mode}', metric] for model in
models]
        plt.bar([x + i * 0.2 for x in range(len(models))], data, width=0.2,
label=mode.upper())
    plt.xticks([x + 0.2 for x in range(len(models))], models)
    plt.title(f'{metric} 指标对比（不同输入模式）')
    plt.ylabel(metric)
    plt.legend()
    plt.tight_layout()
    plt.show()

# ===== 图 4: 输入维度 vs 性能趋势图 =====
input_dims = {'nwp': 8, 'lmd': 6, 'mixed': 14}
for metric in ['C_R', 'Q_R']:
    plt.figure(figsize=(8, 5))
    for model in models:
        y = [df_results.loc[f'{model}_{mode}', metric] for mode in modes]
        x = [input_dims[mode] for mode in modes]
        plt.plot(x, y, marker='o', label=model)
    plt.title(f'{metric} vs 输入特征维度')
    plt.xlabel("输入特征数量")
    plt.ylabel(metric)
    plt.grid(True)
    plt.legend()
    plt.tight_layout()
    plt.show()

if __name__ == "__main__":
    model_dict = {
        # "BiFusionModel": BiFusionModel(input_len=96, input_dim=15),

```



```
# "PureLSTM": PureLSTM(input_len=96, input_dim=15),
# "FusionModel": FusionModel(input_len=96)
}

df_results = run_input_mode_experiments(df, model_dict)
visualize_q4_results(df_results)

# 结束计时
end_time = time.time()
elapsed_time = end_time - start_time

print(f'代码执行时间: {elapsed_time:.4f} 秒')
```